

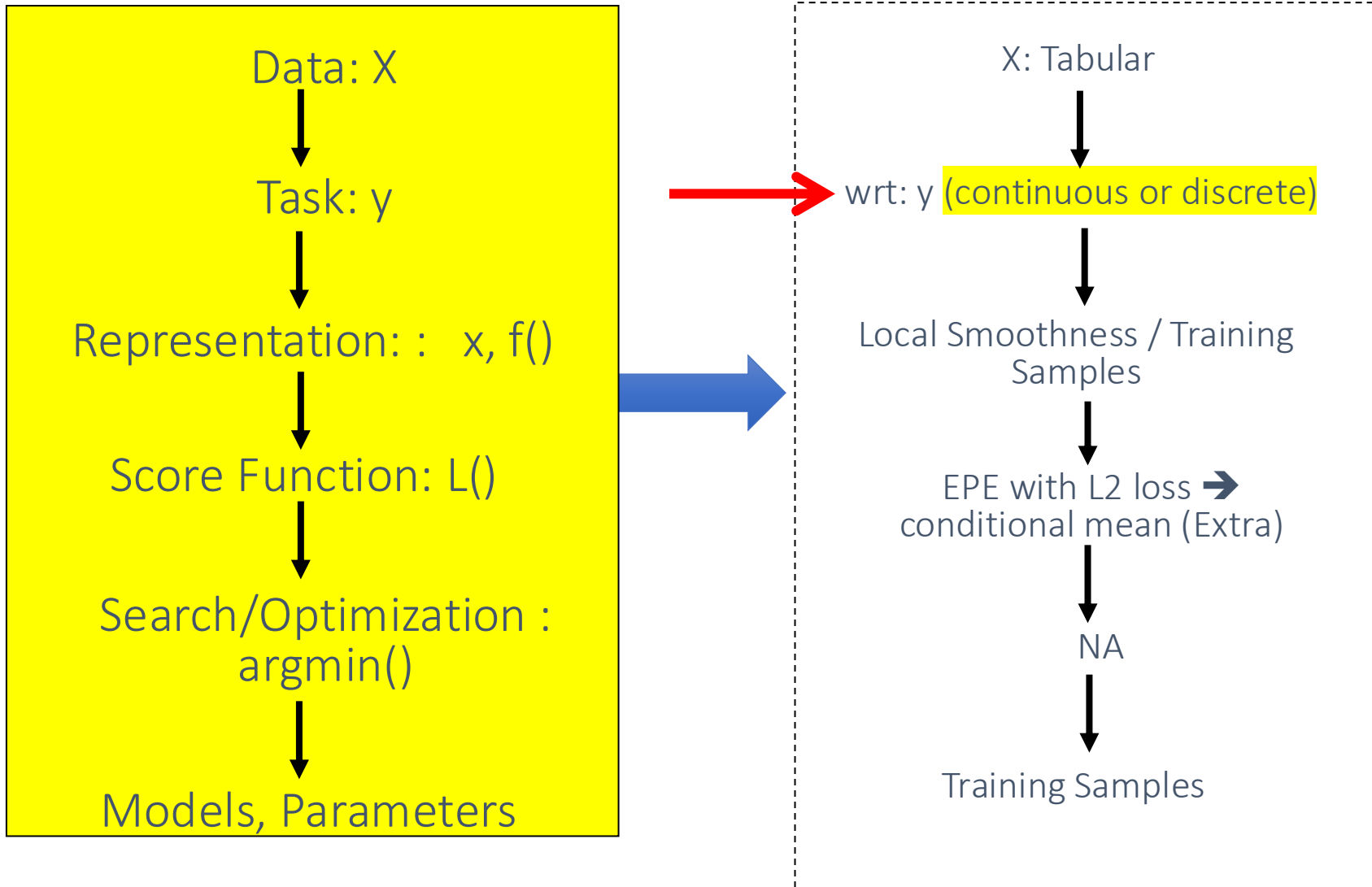
UVA CS 4774: Machine Learning

Lecture 8: K-nearest-neighbor (regressor or classifier)

Dr. Yanjun Qi

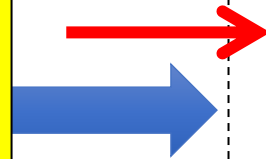
University of Virginia
Department of Computer Science

Today: **K-nearest-neighbor (regressor or classifier)**



Today: **K-nearest-neighbor(regressor or classifier)**

Data: X
↓
Task: y
↓
Representation: $x, f()$
↓
Score Function: $L()$
↓
Search/Optimization : $\text{argmin}()$
↓
Models, Parameters



X : Tabular
↓
Classification or regression
↓
Local Smoothness / Training Samples
↓
EPE with L2 loss → conditional mean (Extra)
↓
NA
↓
Training Samples

X_1	X_2	X_3	Y

1. Tabular Dataset for regression

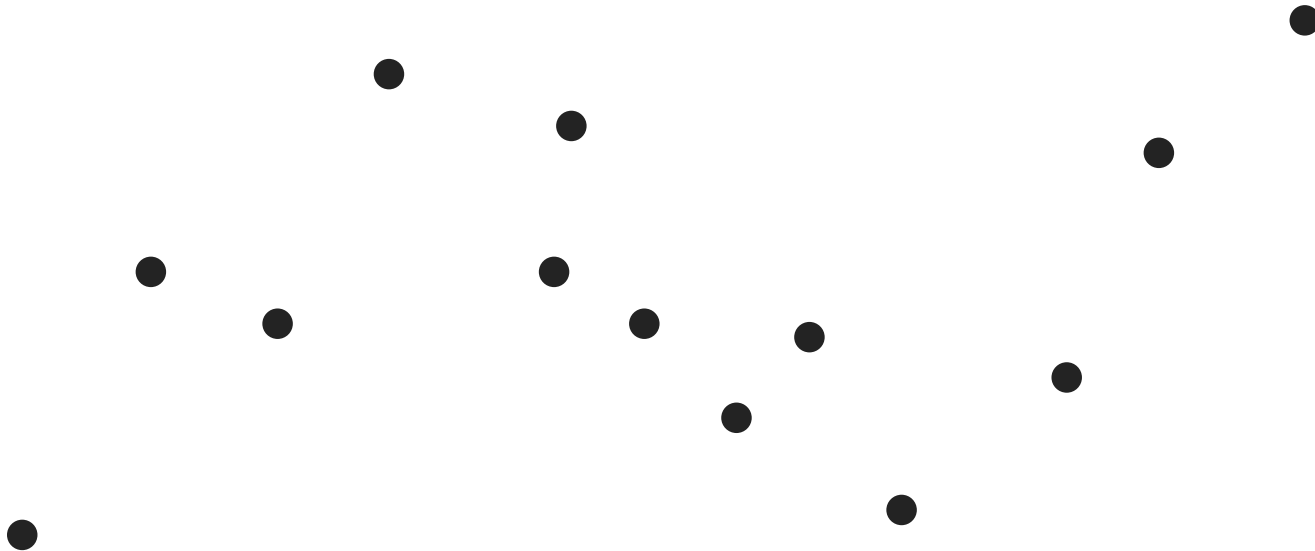
$$f : X \rightarrow Y$$

Output Class:
numerical
variable

- **Data**/points/instances/examples/samples/records: [rows]
- **Features**/attributes/dimensions/independent variables/covariates/predictors/regressors: [columns, except the last]
- **Target**/outcome/response/label/dependent variable: special column to be predicted [last column in the above matrix]

K=5-Nearest Neighbor (1D input) for Regression

TRAINING SET	$\mathcal{D} = \{(x_i, y_i) : i = 1, \dots, n\}$
NEIGHBORS	$k = 5$
DISTANCE	$d(x, x_i) = x - x_i , x \in \mathbb{R}$

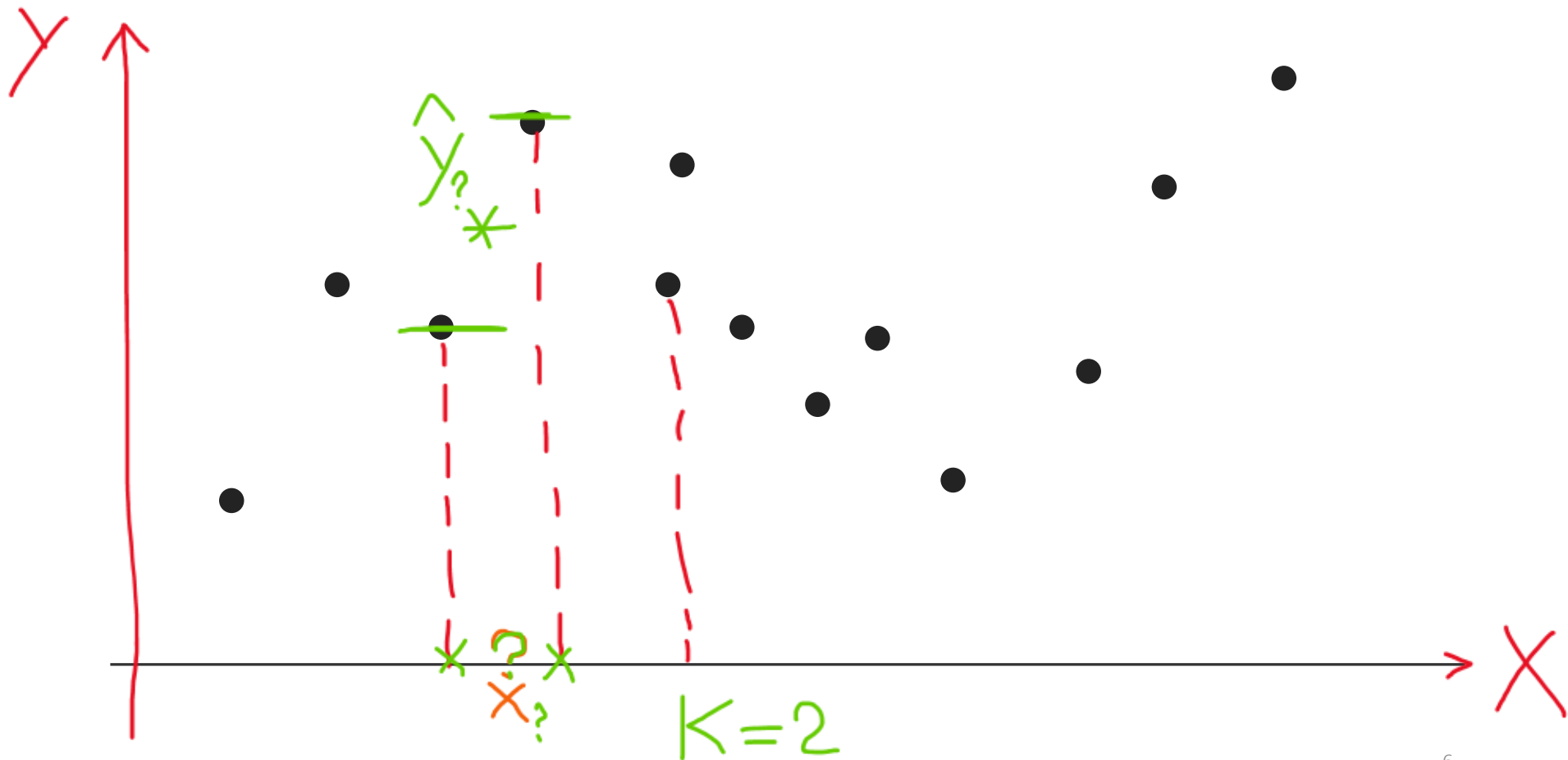


K=5-Nearest
Neighbor (1D
input) for
Regression

TRAINING SET $\mathcal{D} = \{(x_i, y_i) : i = 1, \dots, n\}$

NEIGHBORS **$k = 2$**

DISTANCE $d(x, x_i) = |x - x_i|, x \in \mathbb{R}$



K=5-Nearest Neighbor (1D input) for Regression

TRAINING SET

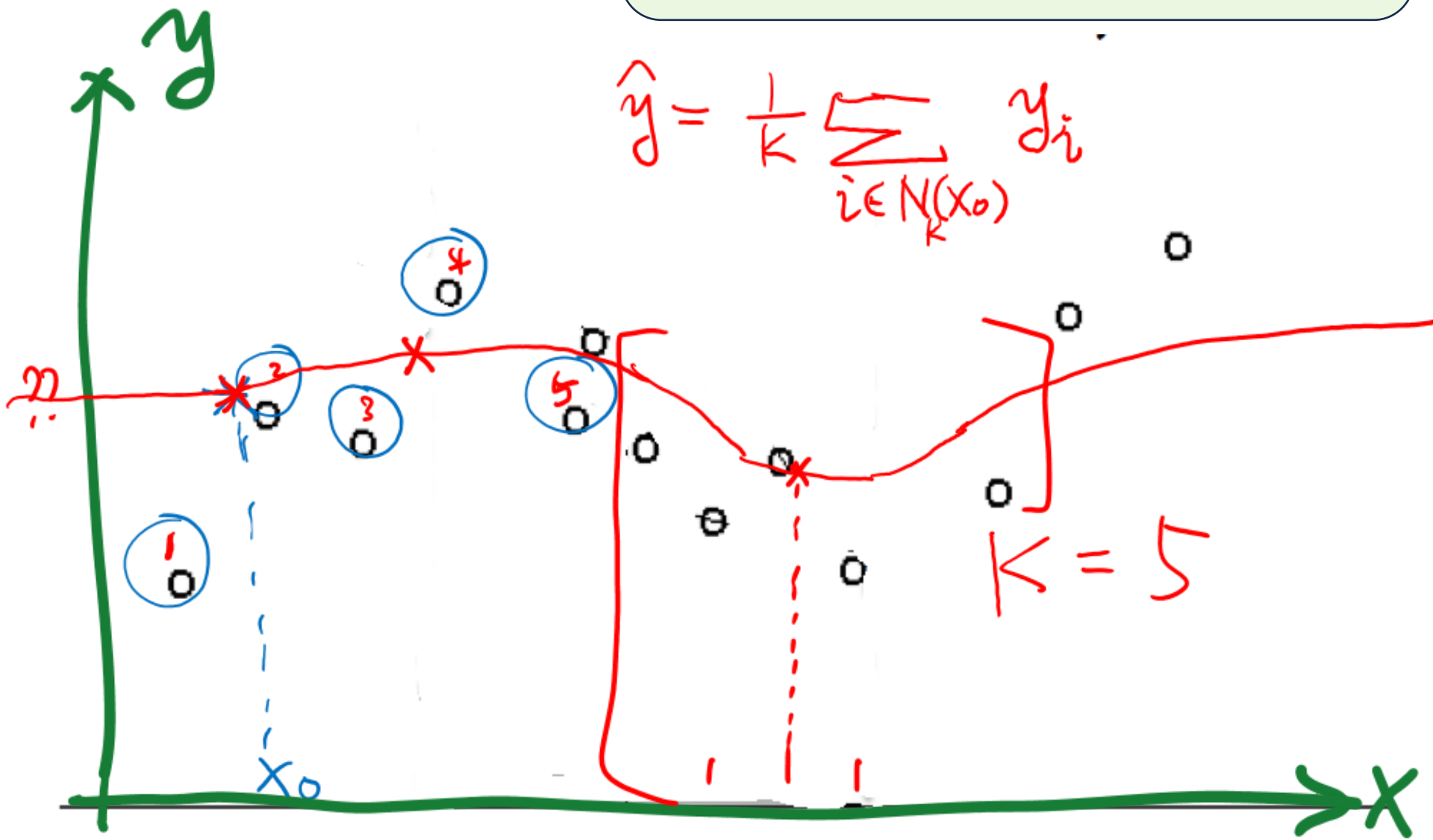
$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$$

NEIGHBORS

$$k = 5$$

DISTANCE

$$d(x, x_i) = |x - x_i|, x \in \mathbb{R}$$



I will code run:

https://colab.research.google.com/drive/1Wzmg5ziMBMvVvLHx_0C4u0x7syF8jPT9?usp=sharing

```
[42] # import regressor
      from sklearn.neighbors import KNeighborsRegressor
      # instantiate with K=5
      knn = KNeighborsRegressor(n_neighbors=5)
      # fit with data
      knn.fit(X, y)
```

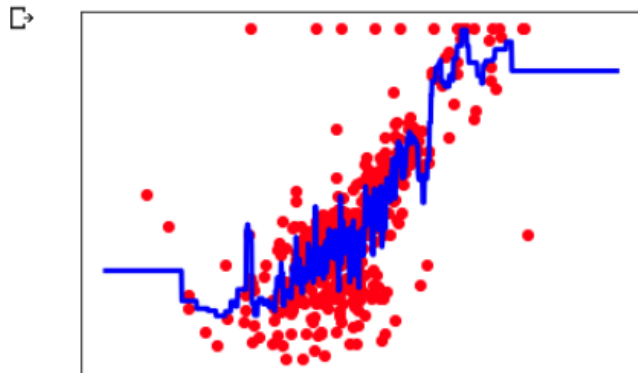
```
↳ KNeighborsRegressor(algorithm='auto', leaf_size=3
                       metric_params=None, n_jobs=No
                       weights='uniform')
```

```
###
Xfit = np.linspace(3, 10, 1000).reshape(-1, 1)
yfit = knn.predict(Xfit)

# Plot outputs
plt.scatter(X, y, color='red')
plt.plot(Xfit, yfit, color='blue', linewidth=3)

plt.xticks(())
plt.yticks(())

plt.show()
```



```
[44] # import regressor
      from sklearn.neighbors import KNeighborsRegressor
      # instantiate with K=5
      knn = KNeighborsRegressor(n_neighbors=100)
      # fit with data
      knn.fit(X, y)
```

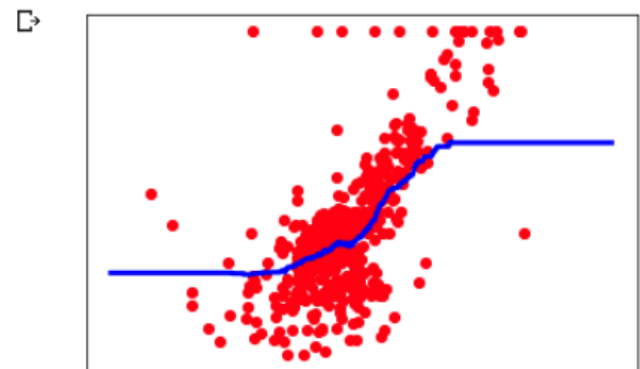
```
↳ KNeighborsRegressor(algorithm='auto', leaf_size=3(
                       metric_params=None, n_jobs=Nor
                       weights='uniform')
```

```
###
Xfit = np.linspace(3, 10, 1000).reshape(-1, 1)
yfit = knn.predict(Xfit)

# Plot outputs
plt.scatter(X, y, color='red')
plt.plot(Xfit, yfit, color='blue', linewidth=3)

plt.xticks(())
plt.yticks(())

plt.show()
```



X_1	X_2	X_3	Y

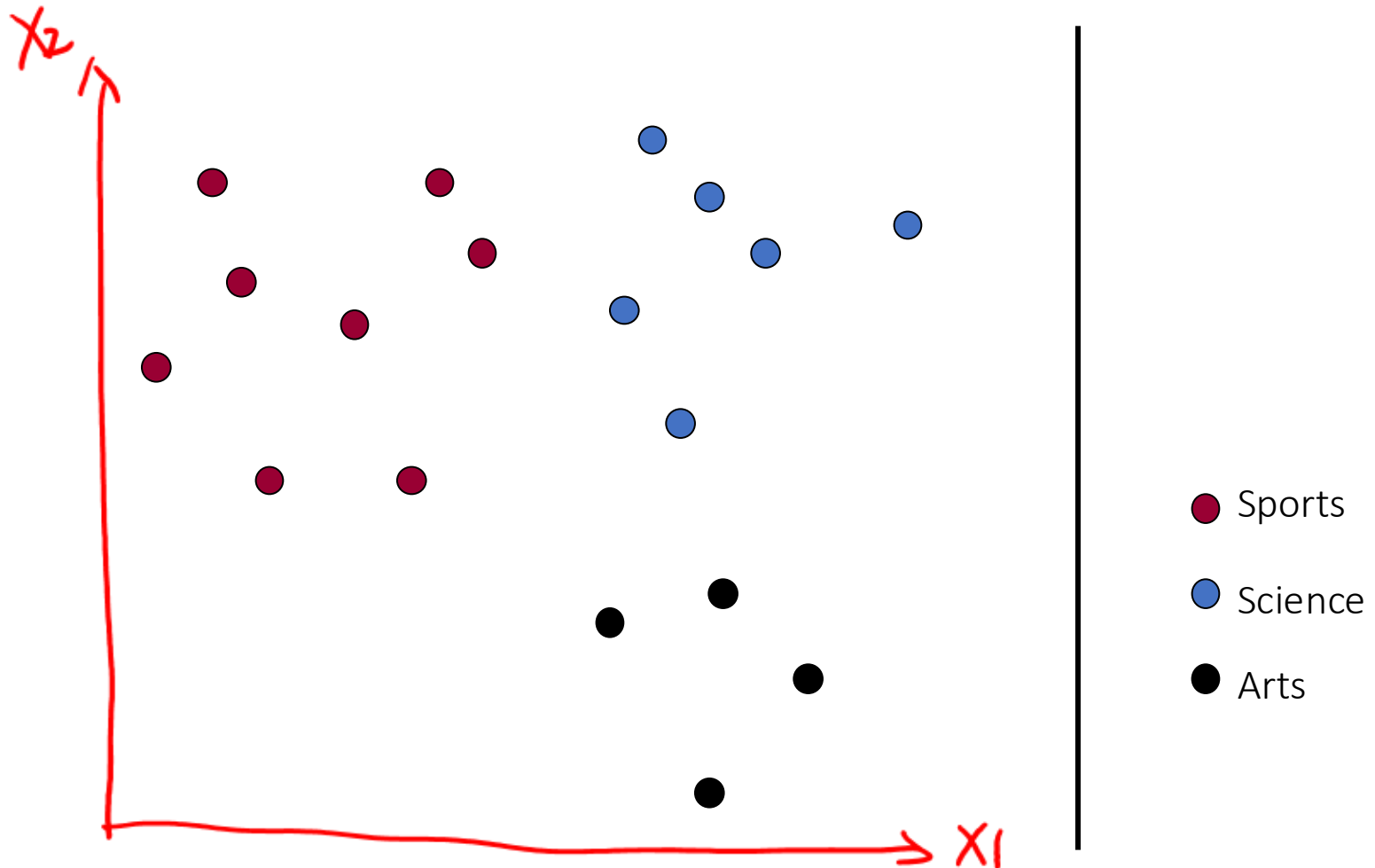
2. Tabular Dataset for **classification**

$$f : \boxed{X} \longrightarrow \boxed{Y}$$

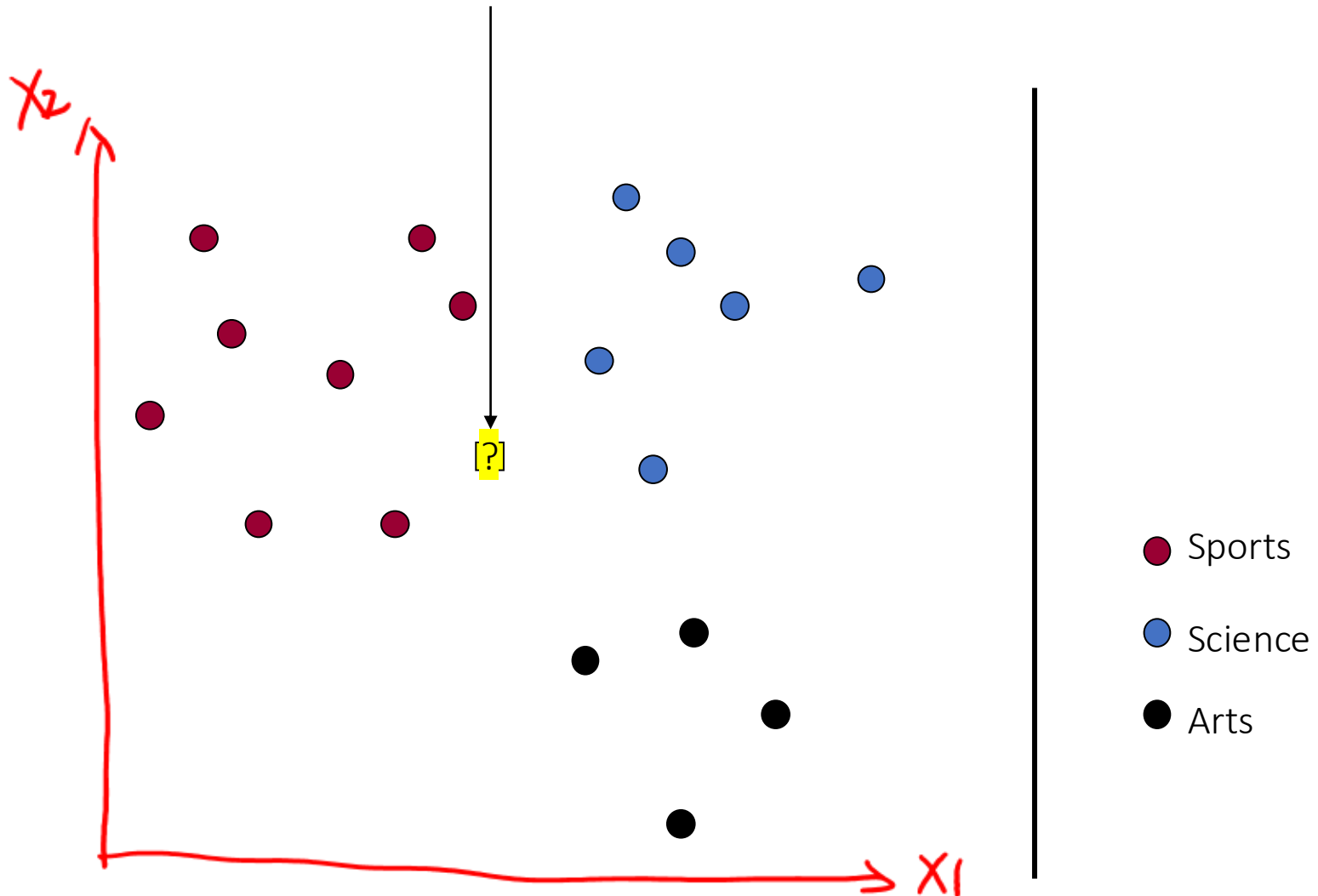
Output Class:
categorical
variable

- **Data**/points/instances/examples/samples/records: [rows]
- **Features**/attributes/dimensions/independent variables/covariates/predictors/regressors: [columns, except the last]
- **Target**/outcome/response/label/dependent variable: special column to be predicted [last column in the above matrix]

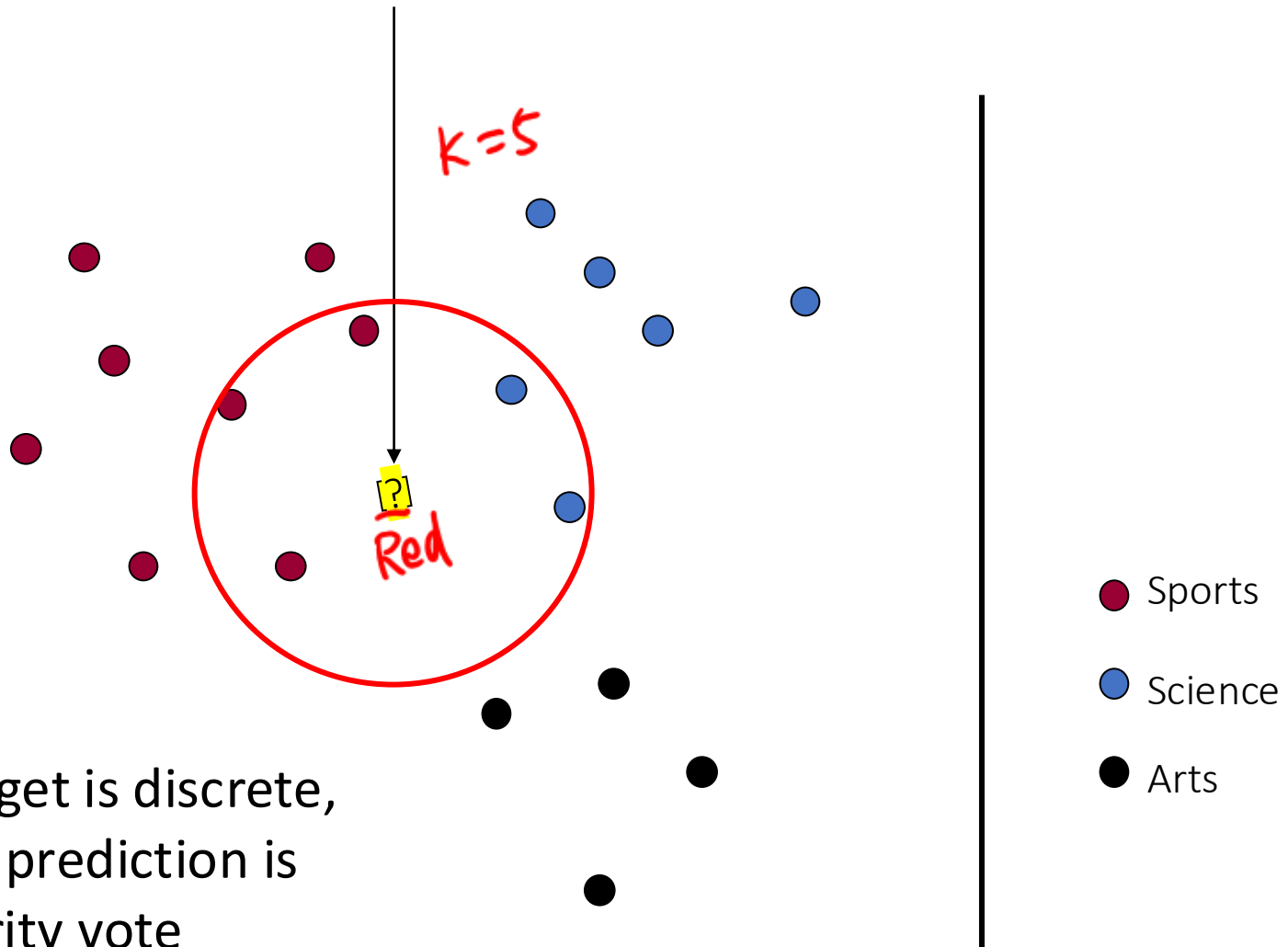
Multiple Target Classes in a Vector Space



Test Document = ?



K-Nearest Neighbor (kNN) classifier



When target is discrete,
the naïve prediction is
the majority vote

K Nearest neighbor (Instance-based method)

It Needs:

1. The set of stored training samples
2. Distance metric to compute distance between samples
3. The value of k , i.e., the number of nearest neighbors to retrieve

K Nearest neighbor (Training Mode)

It Needs:

1. The set of stored training samples
2. Distance metric to compute distance between samples
3. The value of k , i.e., the number of nearest neighbors to retrieve

Training Mode:

- (Naïve) version: DO NOTHING !!!!

K Nearest neighbor (Testing Mode)

It Needs:

1. The set of stored training samples
2. Distance metric to compute distance between samples
3. The value of k , i.e., the number of nearest neighbors to retrieve

Training Mode:

- (Naïve) version: DO NOTHING !!!!

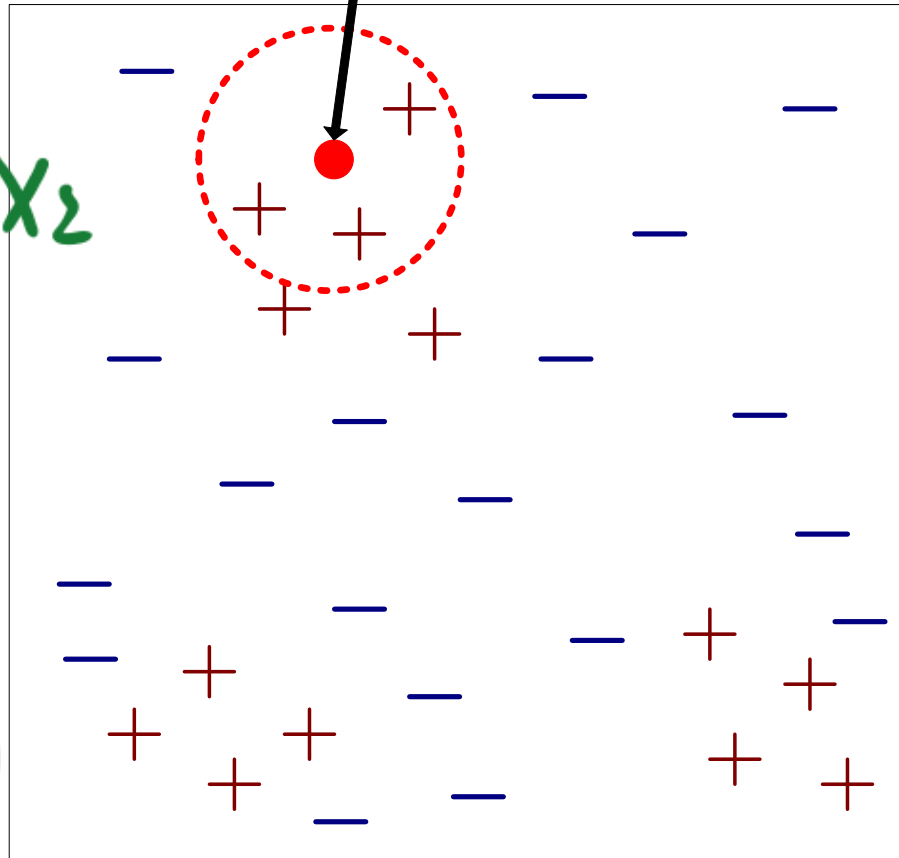
Testing Model: To classify **unknown** sample:

- **Step1**: Compute distance to **all** training records
- **Step2**: Identify **k nearest** neighbors
- **Step3**: Use class labels of nearest neighbors to determine the class label of unknown record (e.g., by taking majority vote)

K Nearest neighbor (Testing/Deploy Mode)

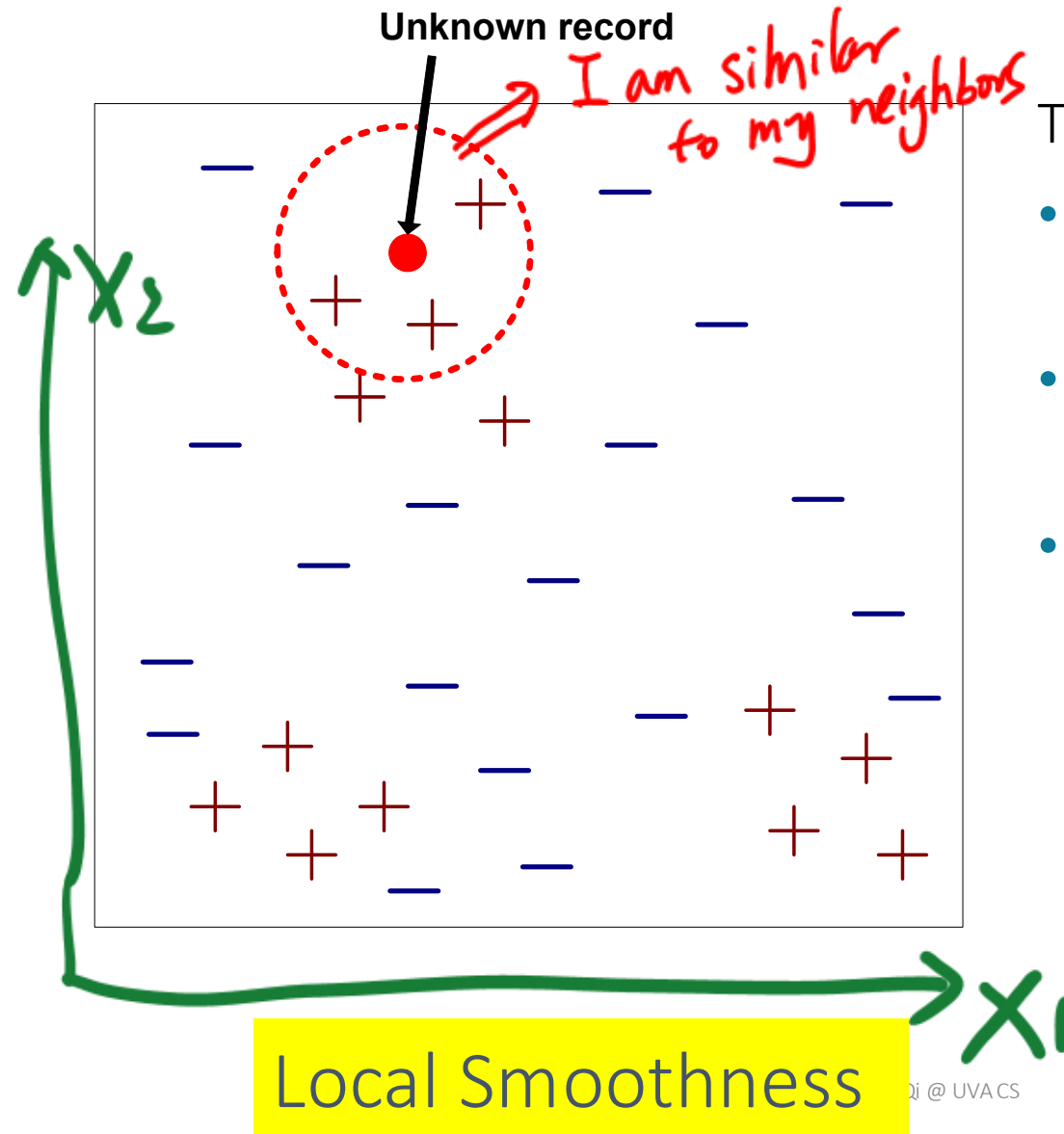
Local Smoothness

Unknown record



To classify **unknown** sample:

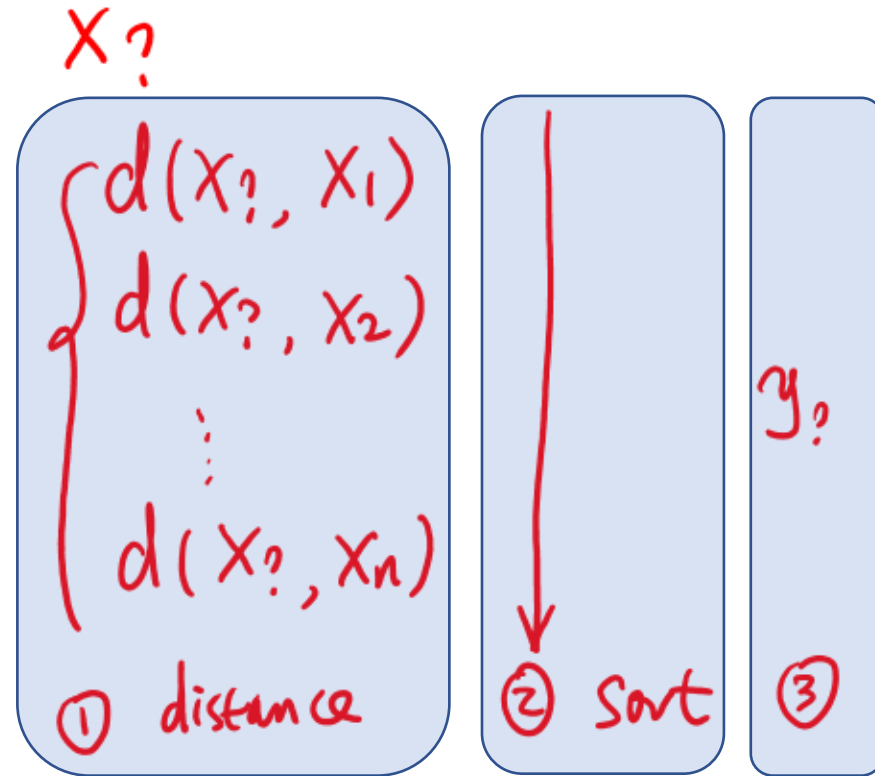
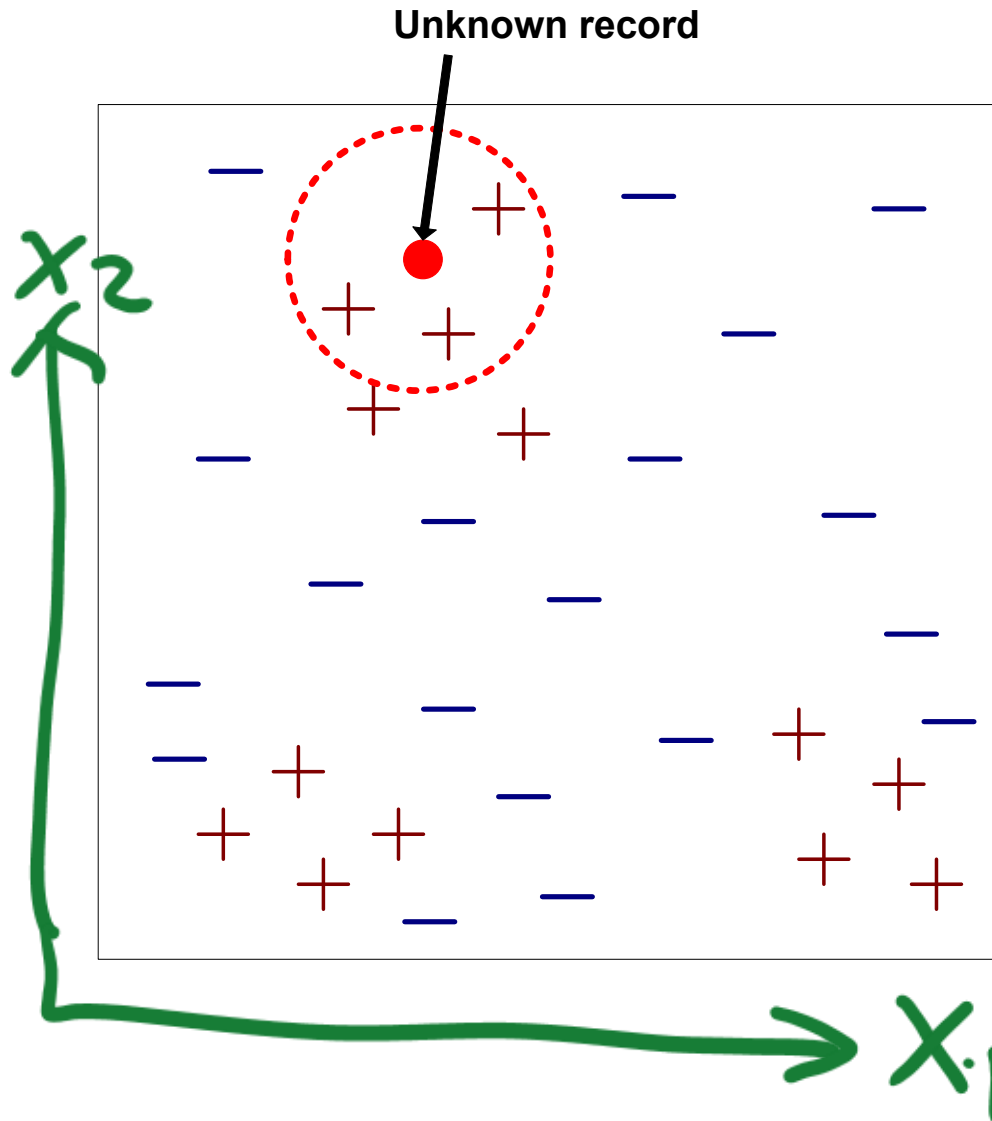
K Nearest neighbor (Testing/Deploy Mode)



$X?$
To classify **unknown** sample:

- **Step1**: Compute distance to all training records
- **Step2**: Identify **k nearest** neighbors
- **Step3**: Use class labels of nearest neighbors to determine the class label of unknown record (e.g., by taking majority vote)

K Nearest neighbor (Testing/Deploy Mode)



$$y \in \{+, -\}$$

	X_1	X_2	X_p	Y
s_1				
s_2				
s_3				
s_4				
s_5				
s_n				
$s_?$				

Step 1: $d(\vec{\tilde{S}}_1, \vec{\tilde{S}}_?)$
 $p \times 1$ $p \times 1$

$O(p)$

$[x_1 \ x_2 \ \dots \ x_p]$ $[x_1 \ \dots \ x_p]$

$$\sum_{j=1}^p (x_j - \bar{x})^2$$

	X_1	X_2	X_p	Y
s_1				
s_2				
s_3				
s_4				
s_5				
s_n				
$s_?$				

Step 1: $d(\vec{s}_1, \vec{s}_?)$
 $p \times 1$ $p \times 1$

$O(p)$

$[x_1 \ x_2 \ \dots \ x_p]$ $[x_1 \ \dots \ x_p]$

$$\sum_{j=1}^p (x_j - x_j^?)^2$$

$\left\{ \begin{array}{l} d(\vec{s}_1, \vec{s}_?) \\ d(\vec{s}_2, \vec{s}_?) \\ \vdots \\ d(\vec{s}_n, \vec{s}_?) \end{array} \right\} \Rightarrow O(np)$

	X_1	X_2	X_p	Y
s_1				
s_2				
s_3				
s_4				
s_5				
s_n				
$s_?$				

Step 1: $d(\vec{s}_1, \vec{s}_?)$
 $p \times 1$ $p \times 1$

$O(p)$

$[x_1 \ x_2 \ \dots \ x_p]$ $[x_1 \ \dots \ x_p]$

$$\sum_{j=1}^p (x_j - x_j^?)^2$$

$\left\{ \begin{array}{l} d(\vec{s}_1, \vec{s}_?) \\ d(\vec{s}_2, \vec{s}_?) \\ \vdots \\ d(\vec{s}_n, \vec{s}_?) \end{array} \right\} \Rightarrow O(np)$

Step 2: $\text{Sort}(n \text{ select } k) \Rightarrow O(n \log n)$

	X_1	X_2	X_p	Y
s_1				
s_2				
s_3				
s_4				
s_5				
s_n				
$s_?$				

Step 1: $d(\vec{s}_1, \vec{s}_?)$ $O(p)$
 $p \times 1$ $p \times 1$

$[x_1 \ x_2 \ \dots \ x_p]$ $[x_1 \ \dots \ x_p]$

$$\sum_{j=1}^p (x_j - x_j^?)^2$$

$\begin{cases} d(\vec{s}_1, \vec{s}_?) \\ d(\vec{s}_2, \vec{s}_?) \\ \vdots \\ d(\vec{s}_n, \vec{s}_?) \end{cases} \Rightarrow O(np)$

Step 2: $\text{Sort}(n \text{ select } k) \Rightarrow O(n \log n)$

When m test $\Rightarrow O(mnp + mn \log n)$

K-Nearest Neighbor: How to decide:

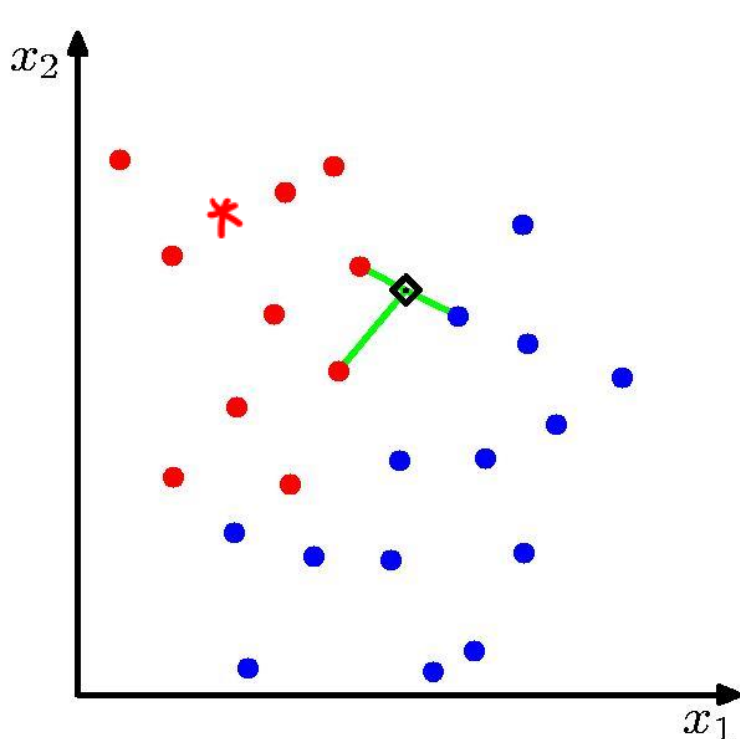
- Decision of output value is delayed till a new instance arrives
- Target variable may be discrete or real-valued
 - When target is discrete, the naïve prediction is the **majority vote**

What if majority gives tie?
What is the tie case?

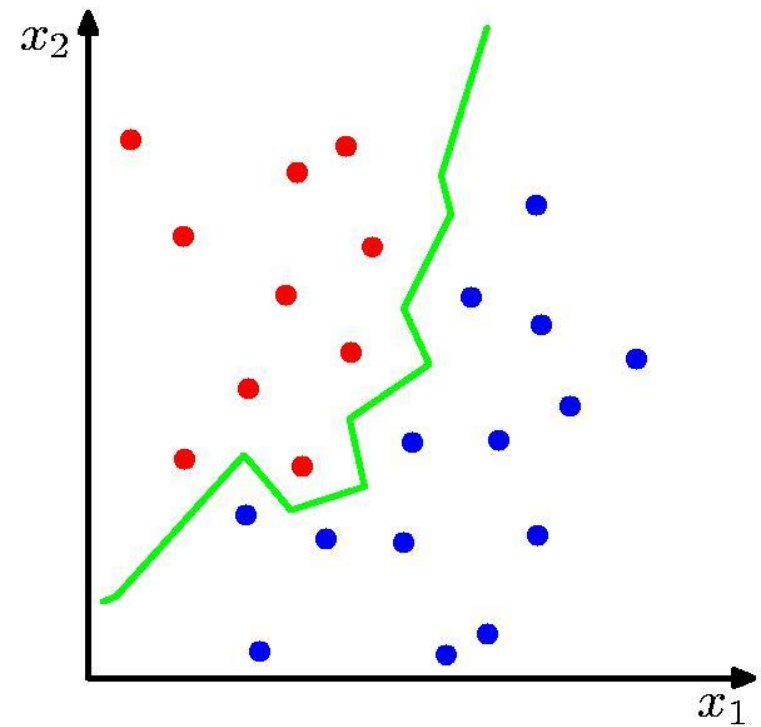
e.g. Decision boundary implemented by 4NN

k-nearest neighbors of a sample x :

➔ datapoints that have the k smallest distances to x among all training points



(a)



(b)

K-Nearest Neighbor: How to decide:

- Decision of output value is delayed till a new instance arrives
- Target variable may be discrete or real-valued
 - When target is discrete, the naïve prediction is the majority vote
 - When target is continuous, the naïve prediction is the mean value of the k nearest training examples

$$\hat{y}(x_0) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x_0)} y_i$$

Summary of Nearest neighbor methods

- For regression, average the predictions of the K nearest neighbors.
- For classification, pick the class with the most votes.
 - How should we break [ties]?
 - E.g., Let the k'th nearest neighbor contribute a count that falls off with k. For example,

$$1 + \frac{1}{2^k}$$

Thank You



UVA CS 4774: Machine Learning

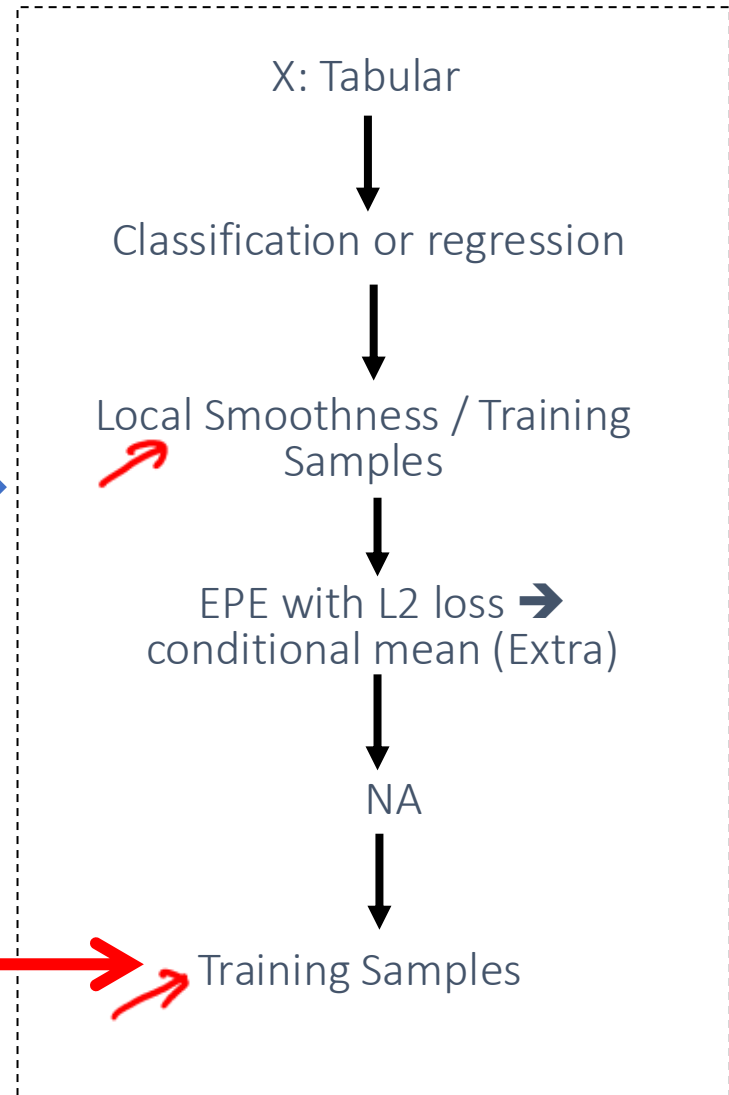
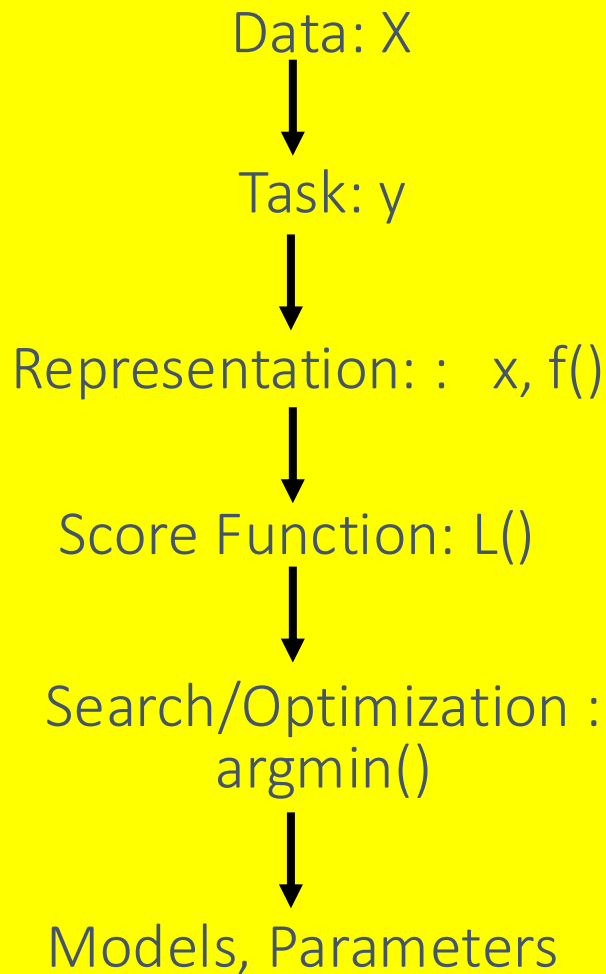
Lecture 8: K-nearest-neighbor (Instance-based Learning)

Module II

Dr. Yanjun Qi

University of Virginia
Department of Computer Science

Today: **K-nearest-neighbor(regressor or classifier)**



We can divide the large variety of supervised classifiers into roughly three major types

1. Discriminative

directly estimate a decision rule/boundary

e.g., support vector machine, decision tree,

e.g. [logistic regression](#), neural networks (NN), deep NN

2. Generative:

build a generative statistical model

e.g., Bayesian networks, [Naïve Bayes classifier](#)



3. Instance based classifiers

- Use observation directly (no models)

- e.g. K nearest neighbors

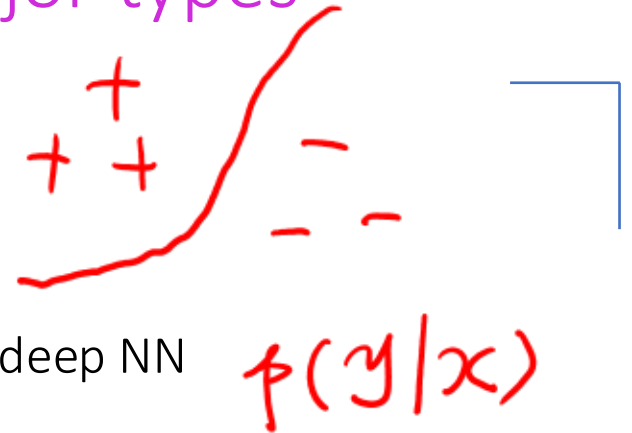
We can divide the large variety of supervised classifiers into roughly three major types

1. Discriminative

directly estimate a decision rule/boundary

e.g., support vector machine, decision tree,

e.g. [logistic regression](#), neural networks (NN), deep NN



2. Generative:

build a generative statistical model

e.g., Bayesian networks, [Naïve Bayes classifier](#)

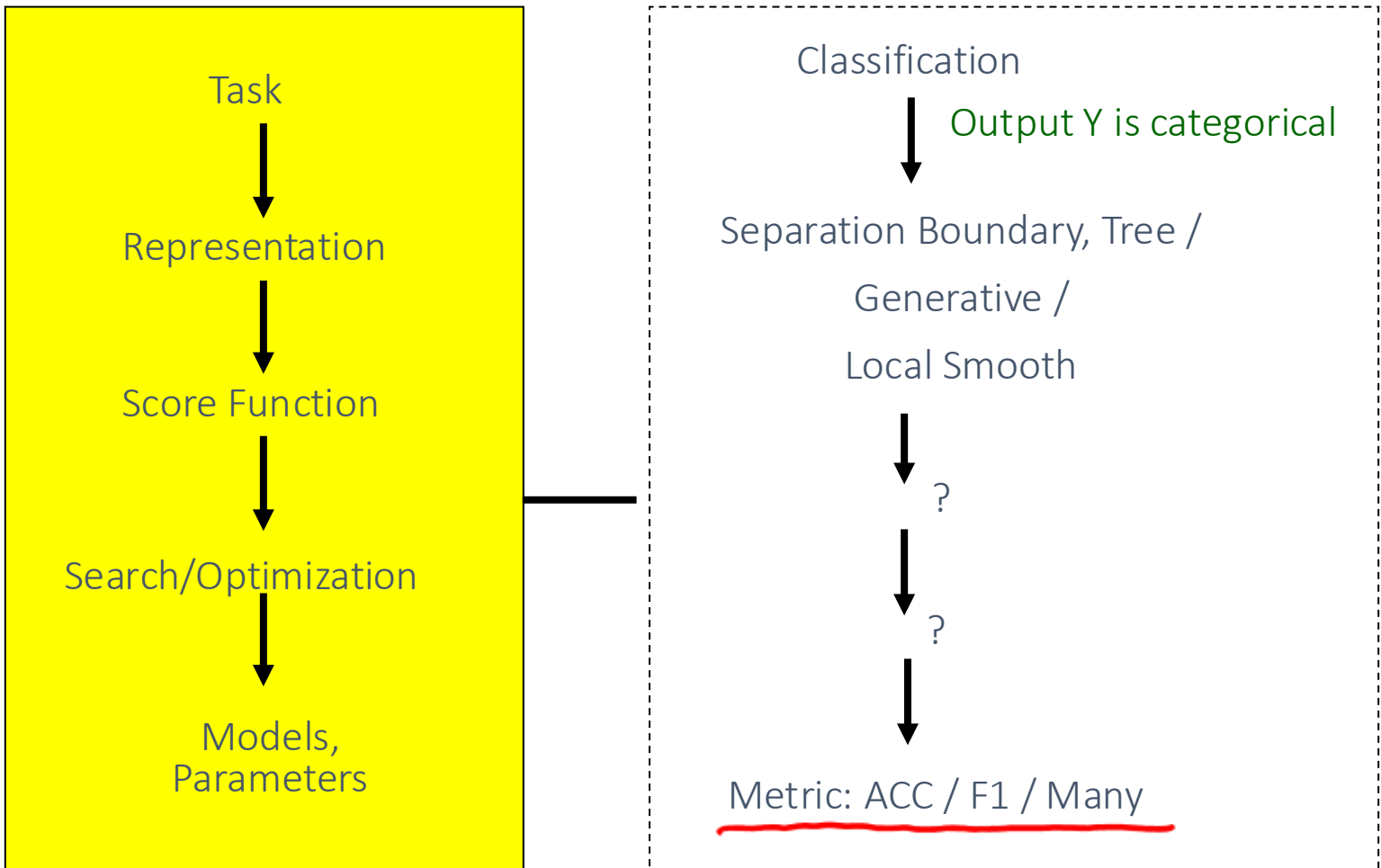
$$P(x|y=c)$$



3. Instance based classifiers

- Use observation directly (no models)
- e.g. K nearest neighbors

Supervised Classifiers



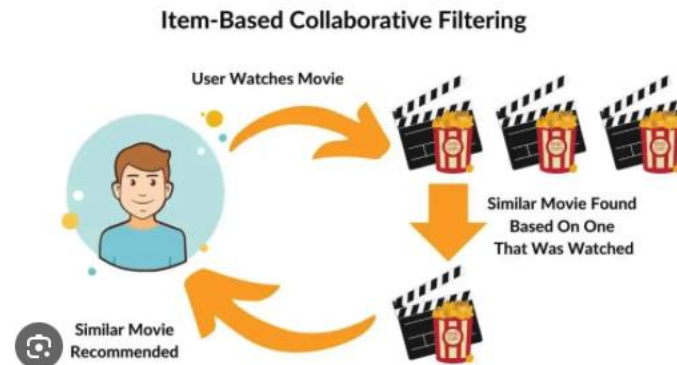
Instance-based Learning

- Simplest form of learning:
 - Training instances are searched for those that most closely resembles new instance
 - The instances themselves represent the knowledge
- Similarity function defines what's “learned”
- Instance-based learning is *lazy* learning

Instance-based Learning Examples:

- K-Nearest Neighbor Algorithm
- Weighted Regression
- Case-based reasoning

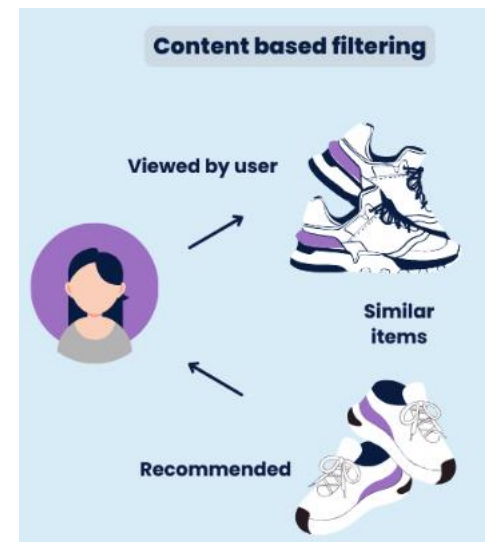
(deep) metric learning $\Rightarrow$ to learn distance fun from data



What makes an Instance-Based Learner?



- A distance metric
- How many nearby neighbors to look at?
- A weighting function (optional)



Popular Distance Metric

- Euclidean
$$D(x, x') = \sqrt{\sum_i \sigma_i^2 (x_i - x_i')^2}$$

- Or equivalently,

$$D(x, x') = \sqrt{(x - x')^T \Sigma (x - x')}$$

- Other metrics:

- L_1 norm: $|x - x'|$
- L_∞ norm: $\max |x - x'|$ (elementwise ...)
- Mahalanobis: where S is full, and symmetric
- Correlation
- Angle
- Hamming distance, Manhattan distance
- ...

String Search

Feature Scaling in Nearest neighbor method

- Scaling issues
 - Attributes may have to be scaled to prevent distance measures from being dominated by one of the attributes
 - Example:
 - height of a person may vary from 1.5 m to 1.8 m $k=1$
 - weight of a person may vary from 90 lb to 300 lb $k=2$
 - income of a person may vary from \$10K to \$1M $k=3$

negative inner product

$$-\sum_{k=1}^p (\bar{x}_{ik} \cdot \bar{x}_{jk}) = d(\bar{x}_i, \bar{x}_j)$$

The relative scaling in the distance metric affect region shapes.

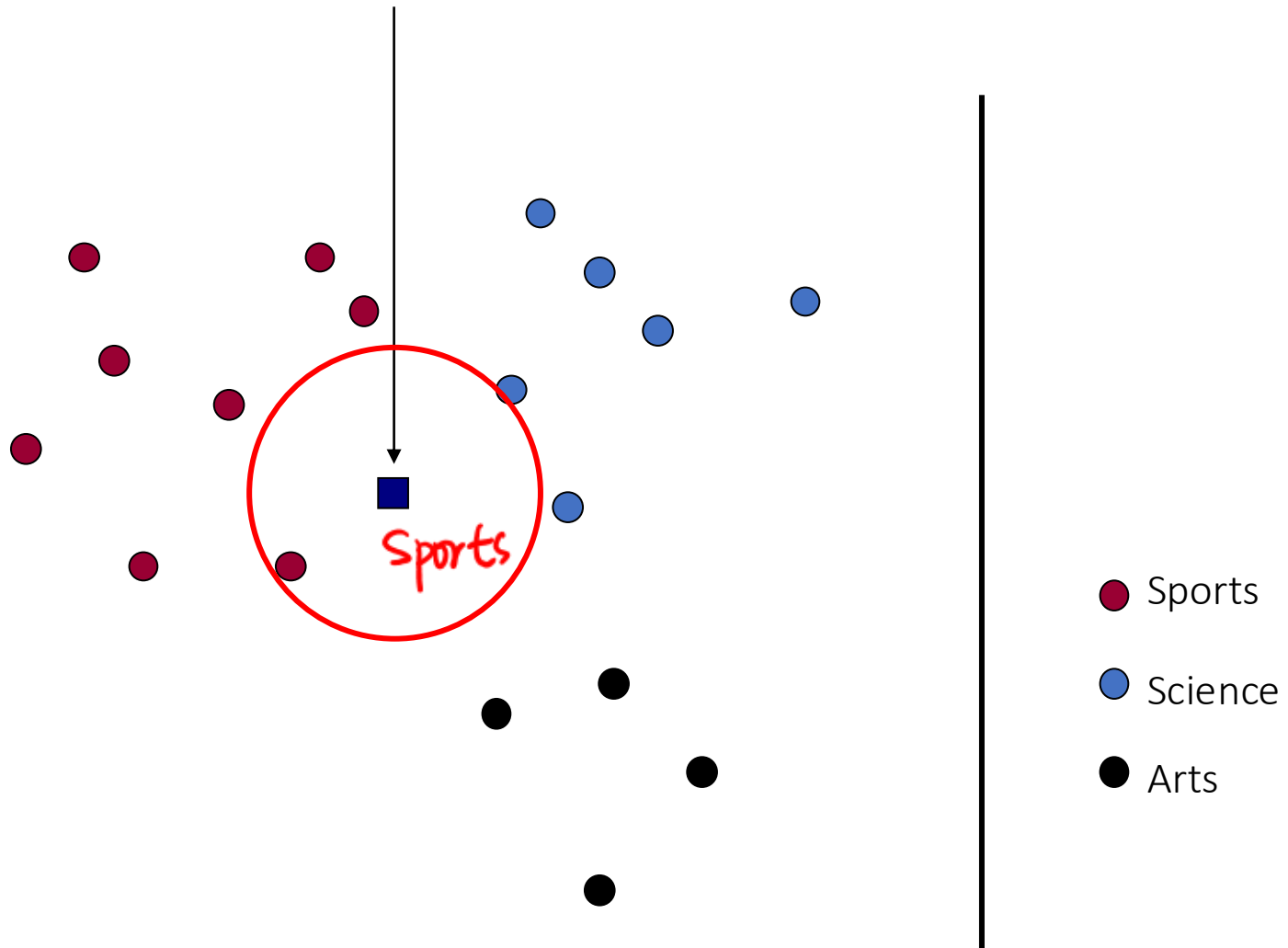
What makes an Instance-Based Learner?

- A distance metric

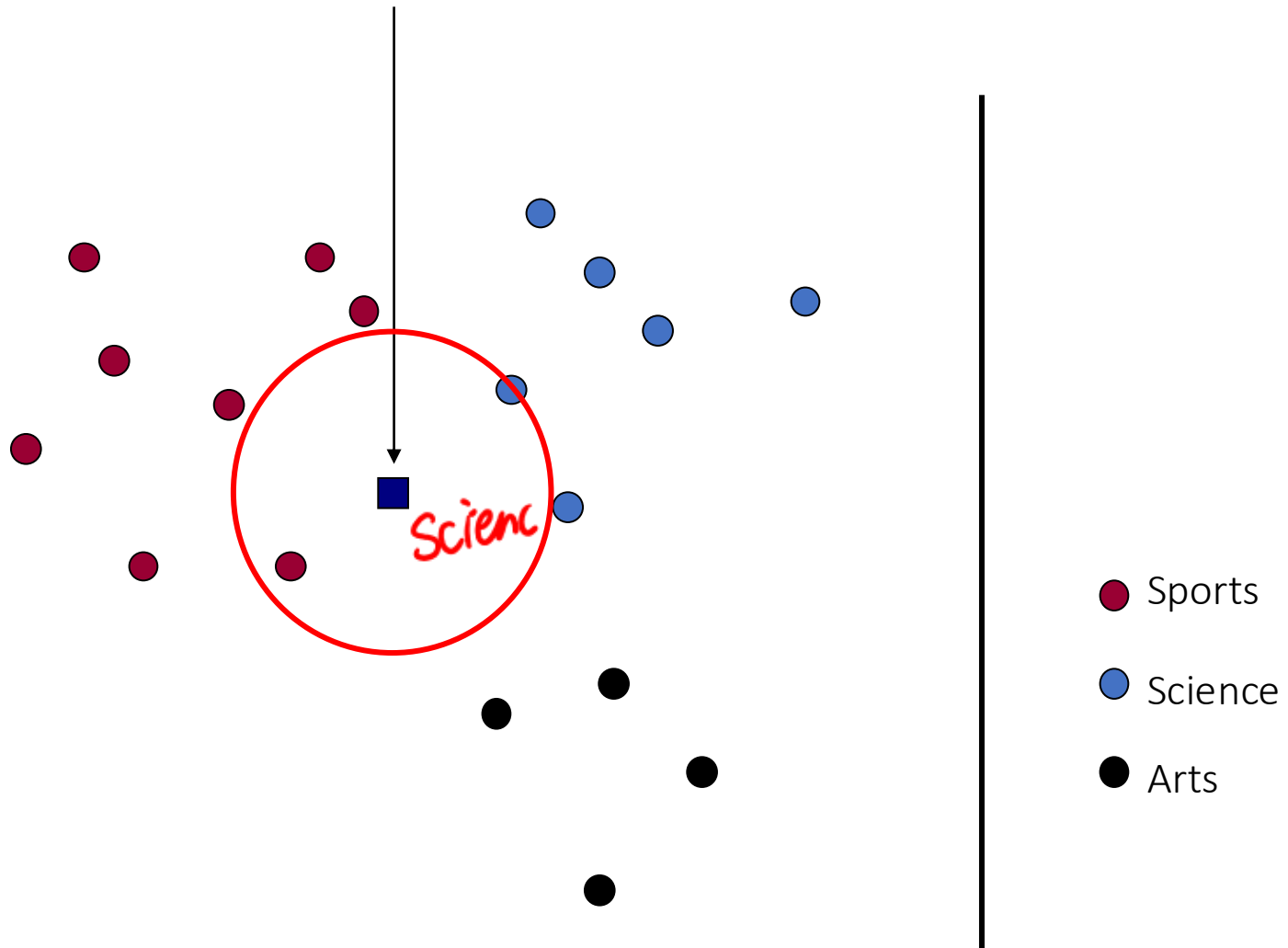
- 
- How many nearby neighbors to look at?

- A weighting function (optional)

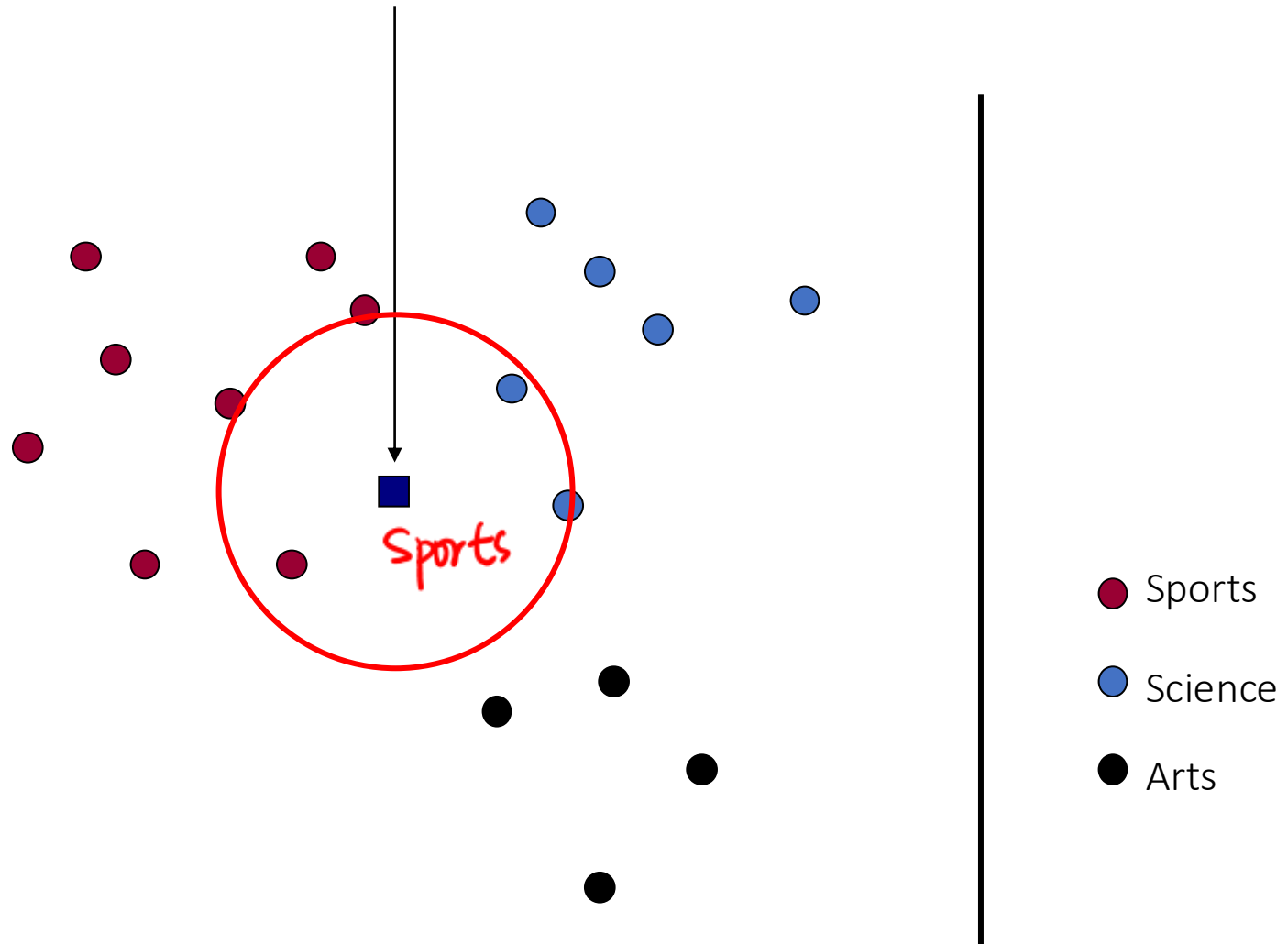
1-Nearest Neighbor (kNN) classifier



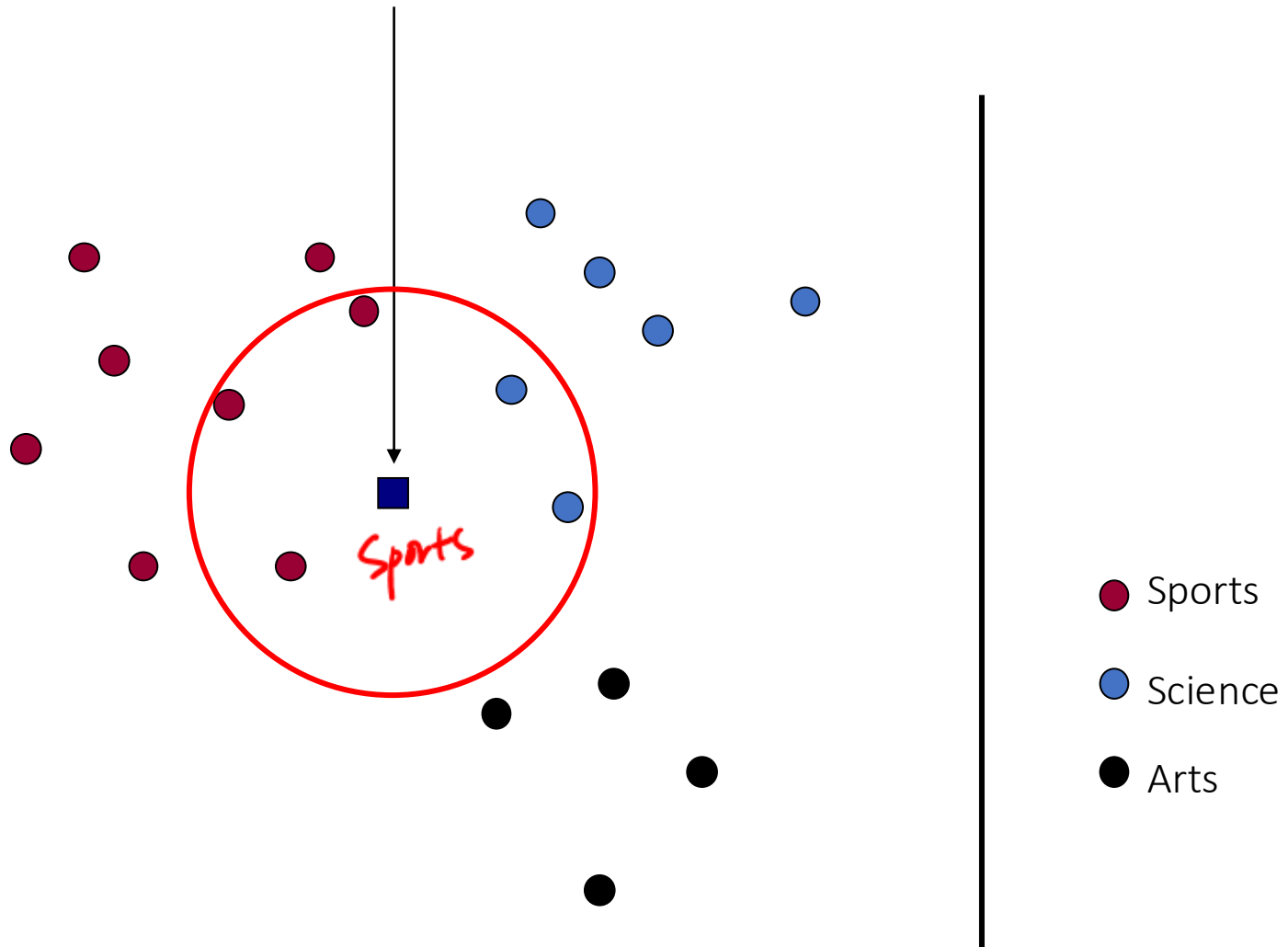
2-Nearest Neighbor (kNN) classifier



3-Nearest Neighbor (kNN) classifier



5-Nearest Neighbor (kNN) classifier



Our code run:

https://colab.research.google.com/drive/1Wzmg5ziMBMvVvLHx_0C4u0x7syF8jPT9?usp=sharing

```
[42] # import regressor
      from sklearn.neighbors import KNeighborsRegressor
      # instantiate with K=5
      knn = KNeighborsRegressor(n_neighbors=5)
      # fit with data
      knn.fit(X, y)
```

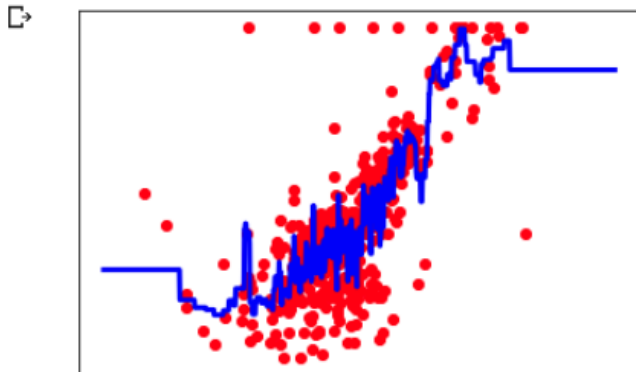
```
↳ KNeighborsRegressor(algorithm='auto', leaf_size=3
                       metric_params=None, n_jobs=No
                       weights='uniform')
```

```
###
Xfit = np.linspace(3, 10, 1000).reshape(-1, 1)
yfit = knn.predict(Xfit)

# Plot outputs
plt.scatter(X, y, color='red')
plt.plot(Xfit, yfit, color='blue', linewidth=3)

plt.xticks(())
plt.yticks(())

plt.show()
```



```
[44] # import regressor
      from sklearn.neighbors import KNeighborsRegressor
      # instantiate with K=5
      knn = KNeighborsRegressor(n_neighbors=100)
      # fit with data
      knn.fit(X, y)
```

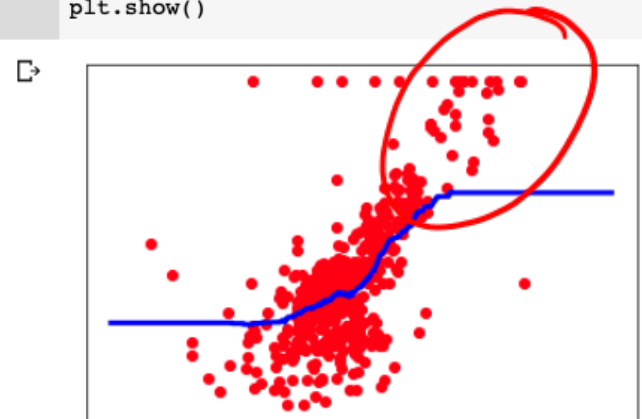
```
↳ KNeighborsRegressor(algorithm='auto', leaf_size=3(
                       metric_params=None, n_jobs=Nor
                       weights='uniform')
```

```
###
Xfit = np.linspace(3, 10, 1000).reshape(-1, 1)
yfit = knn.predict(Xfit)

# Plot outputs
plt.scatter(X, y, color='red')
plt.plot(Xfit, yfit, color='blue', linewidth=3)

plt.xticks(())
plt.yticks(())

plt.show()
```



Model Selection for Nearest neighbor classification

- Choosing the value of k :
 - If k is too small, sensitive to noise points
 - If k is too large, neighborhood may include points from other classes

$k \downarrow$ flexible / varies a lot
 $k \uparrow$ Smooth / varies little

Less complex

More complex



① Polynomial Regression

d

$d \downarrow$

$d \uparrow$

② Ridge Reg

λ
 $\lambda > 0$

$\lambda \uparrow$

$\lambda \downarrow$

③ KNN Reg

k

$k \uparrow$

$k \downarrow$



Polynomial Regression

$d \nearrow$ Complexity $\nearrow$
 $d \searrow$ Complexity $\searrow$

Ridge Regression

$\lambda \nearrow$ Complexity $\searrow$
 $\lambda \searrow$ Complexity $\nearrow$

KNN

$k \nearrow$ Complexity $\searrow$
 $k \searrow$ Complexity $\nearrow$

poly Reg

$d \nearrow$

Ridge

$\lambda \searrow$

kNN

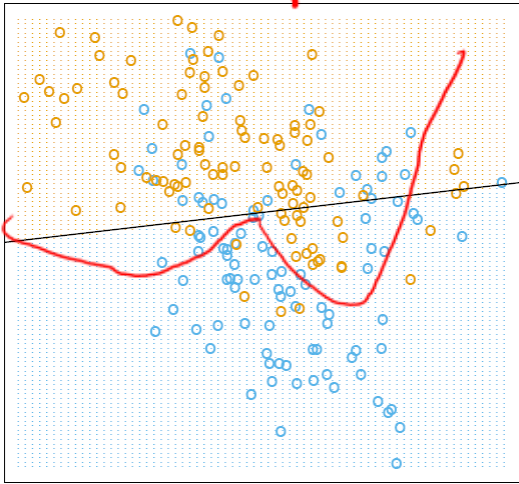
$k \searrow$

model
complex
small

model complexity
large

Decision boundaries in Linear vs. kNN models (Later)

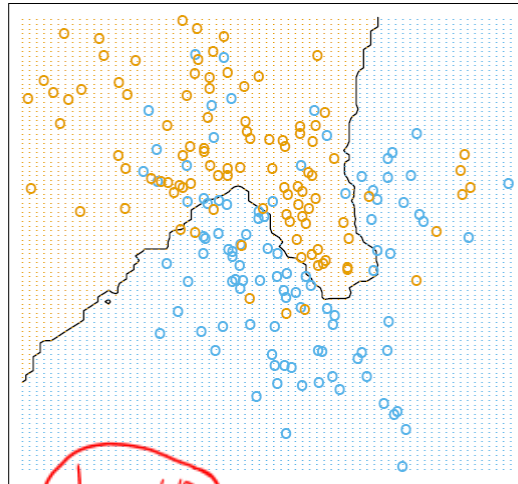
underfit



Low Variance /
High Bias

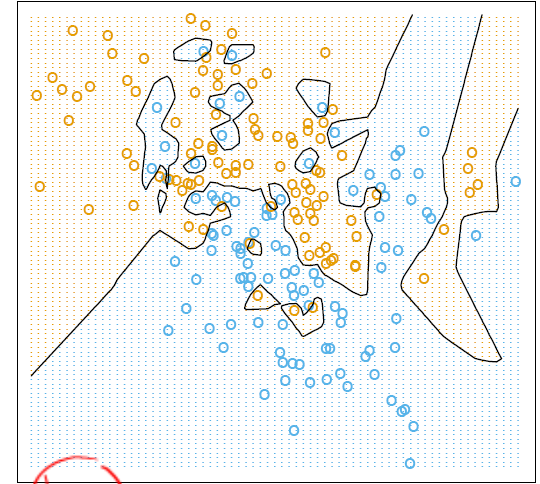
linear regression

- global
- stable
- can be inaccurate



15-nearest neighbor

overfit



1-nearest neighbor

KNN

- local
- accurate
- unstable

Low Bias
/ High Variance

What ultimately matters: GENERALIZATION

Model Selection for Nearest neighbor classification

- Choosing the value of k:
 - If k is too small, sensitive to noise points
 - If k is too large, neighborhood may include points from other classes

- Bias and variance tradeoff

- A small neighborhood $\rightarrow$ large variance $\rightarrow$ [unreliable] estimation

- A large neighborhood $\rightarrow$ large bias $\rightarrow$ [inaccurate] estimation

overfit

underfit

Thank You



What makes an Instance-Based Learner?

- A distance metric
- How many nearby neighbors to look at?
-  • A weighting function (optional)

Nearest neighbor Variation (1)

- Options for determining the class from nearest neighbor list:
Weight the votes according to distance
 - example: weight factor $w = 1 / d^2$

Spurious or less relevant points need to be downweighted

WEIGHTED PREDICTION

$$\hat{y}(x_0) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x_0)} w_i y_i$$

INVERSE-DISTANCE WEIGHT

$$w_i = \frac{1}{d(x_i, x_0)^2}$$

Distance-Weighted k-Nearest Neighbor Algorithm

- Assign weights to the neighbors based on their “distance” from the query point
 - Weight “may” be inverse square of the distances $w = 1 / d^2$
- **Extreme Option:** All training points may influence a particular instance
 - E.g., Shepard’s method/ Modified Shepard, ... by Geospatial Analysis

$$w_i = \frac{1}{d(x_i, x_0)^2}$$

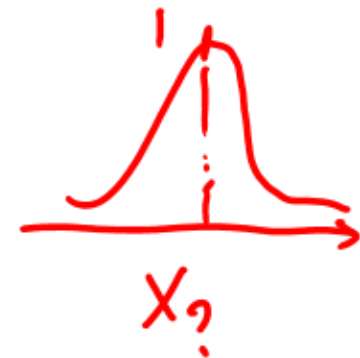
Nearest neighbor Variation (2): RBF kernel Weighted kNN

$$w(\mathbf{x}, \mathbf{x}_i) = \exp \left(-\lambda \|\mathbf{x} - \mathbf{x}_i\|_2^2 \right)$$

For Classification

$$\Pr(y|\mathbf{x}) = \frac{\sum_{i=1}^n w(\mathbf{x}, \mathbf{x}_i) \delta(y, y_i)}{\sum_{i=1}^n w(\mathbf{x}, \mathbf{x}_i)}$$

$$\delta(y, y_i) = \begin{cases} 1 & y = y_i \\ 0 & y \neq y_i \end{cases}$$



For Regression

$$\hat{y} = \frac{\sum w(\mathbf{x}, \mathbf{x}_i) y_i}{\sum w(\mathbf{x}, \mathbf{x}_i)} \quad \text{weighted regression}$$

Today: **K-nearest-neighbor(regressor or classifier)**

Data: X
↓
Task: y
↓
Representation: $x, f()$
↓
Score Function: $L()$
↓
Search/Optimization : $\text{argmin}()$
↓
Models, Parameters



X : Tabular
↓
Classification or regression
↓
Local Smoothness / Training Samples
↓
EPE with L2 loss → conditional mean (Extra)
↓
NA
↓
Training Samples as model / to Select k



We aim to make our trained model

- 1. Generalize Well
- 2. Computational Scalable and Efficient
- 3. Trustworthy: Robust / Interpretable
 - Especially for some domains, this is about trust!

Today: **K-nearest-neighbor(regressor or classifier)**

Data: X
↓
Task: y
↓
Representation: $x, f()$
↓
Score Function: $L()$
↓
Search/Optimization : $\text{argmin}()$
↓
Models, Parameters



X : Tabular
↓
Classification or regression
↓
Local Smoothness / Training Samples
↓
EPE with L2 loss → conditional mean (Extra)
↓
NA
↓
Training Samples

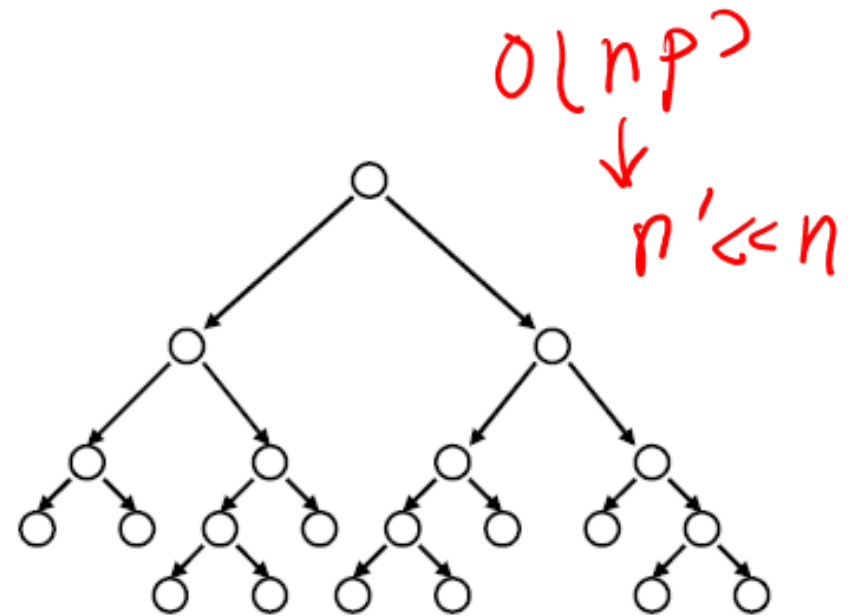
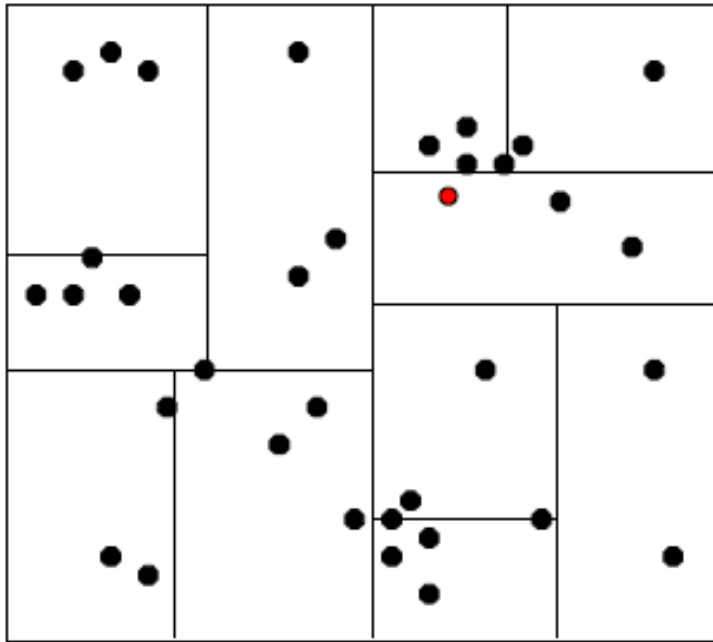
Computational
Scalable?

Computational Time Cost

	Train (n)	Test ($m=1$) $\hat{y} = \beta^T x$
Linear Regression	$O(np^2 + p^3)$	$O(p)$
KNN Reg	$O(1)$	$O(np) +$ $O(\text{sort } n-k)$???

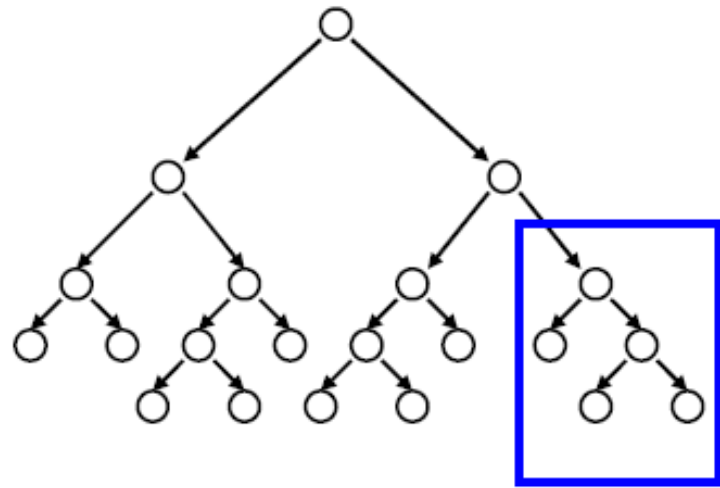
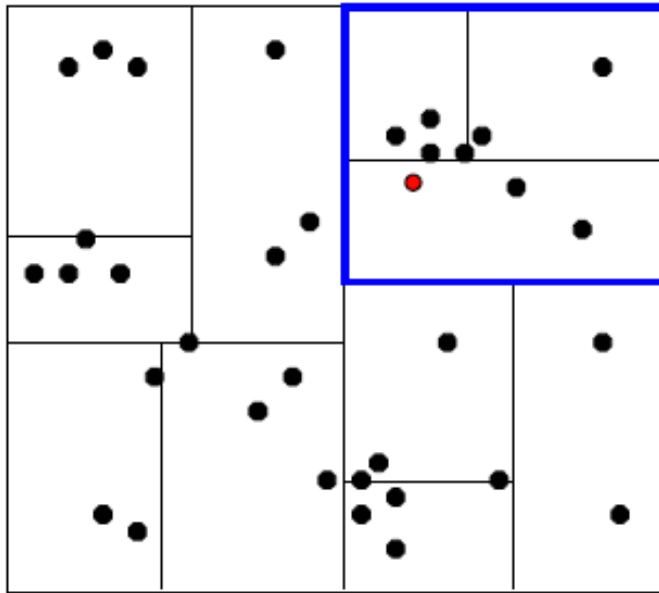
$$p = 30,000$$
$$n = 20,000$$

NN Search by KD Tree



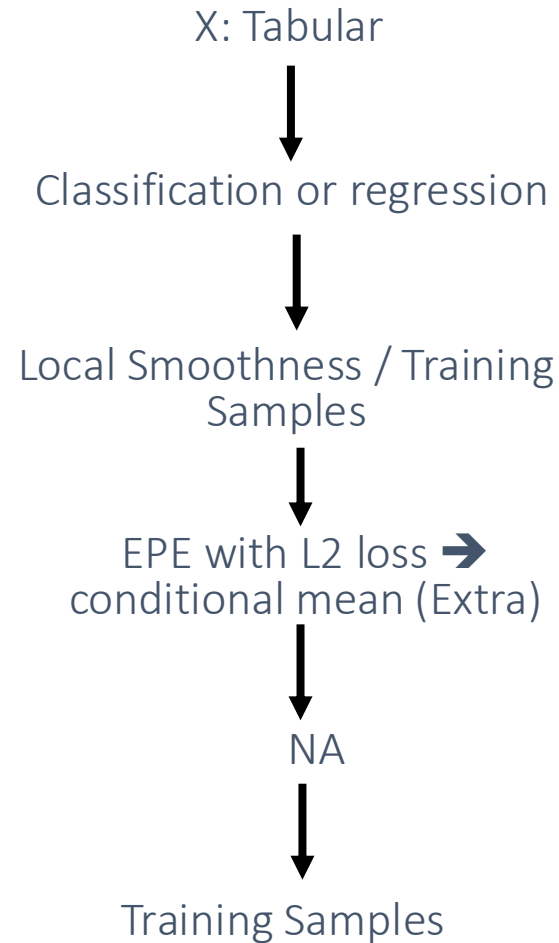
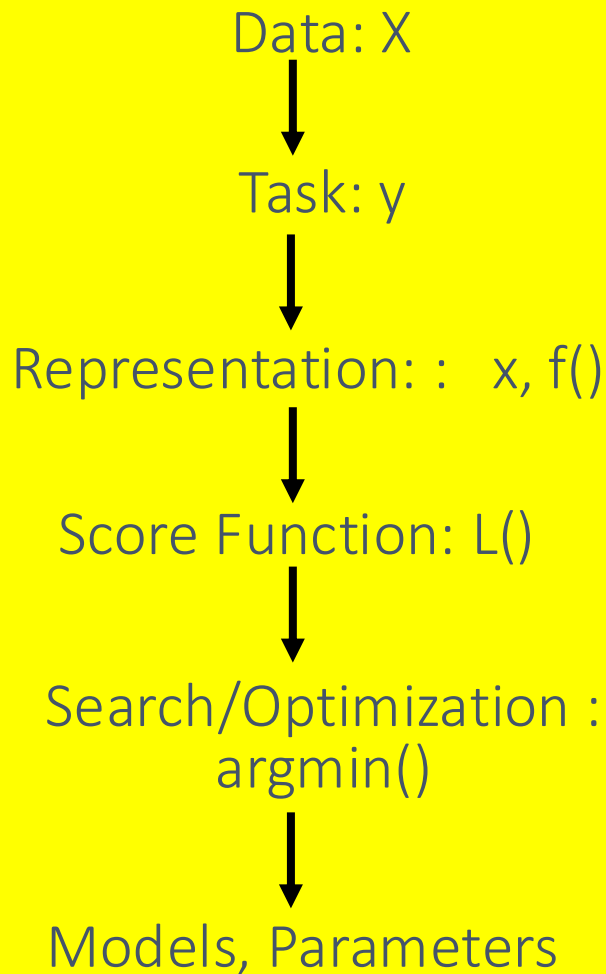
We traverse the tree looking for the nearest neighbor of the query point.

NN Search by KD Tree



Examine nearby points first: Explore the branch of the tree that is closest to the query point first.

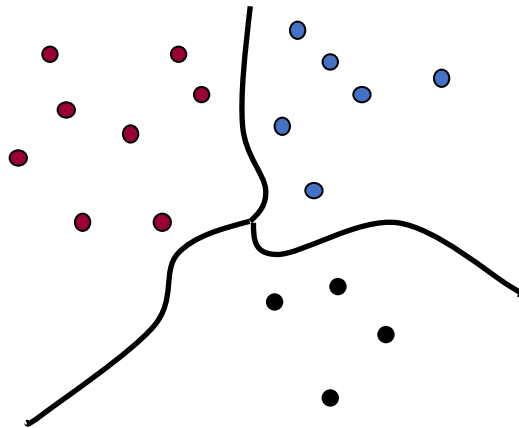
Today: **K-nearest-neighbor(regressor or classifier)**



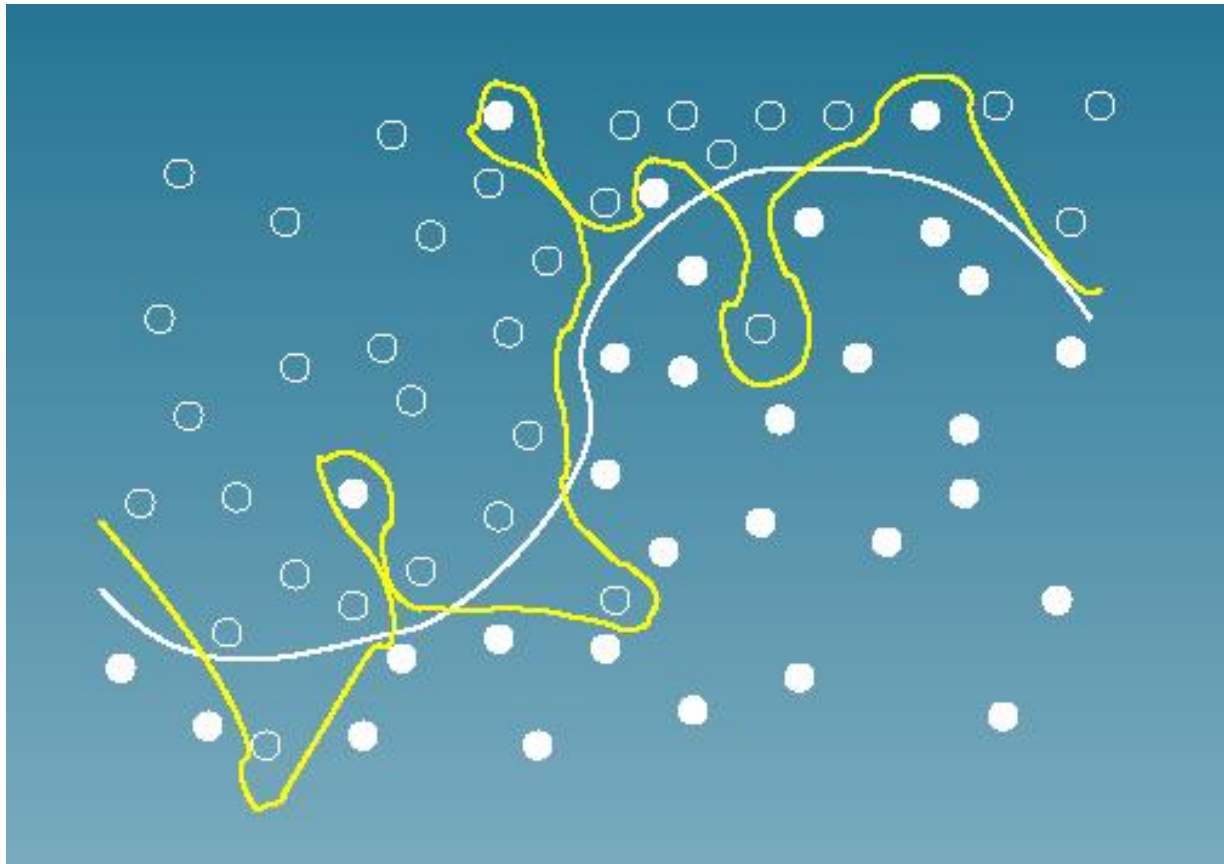
Asymptotically Sound?

kNN Is Close to Optimal Asymptotically (EXTRA, when infinite training)

- Cover and Hart 1967
- Asymptotically, the error rate of 1-nearest-neighbor classification is less than twice the Bayes rate [error rate of classifier knowing model that generated data]
- In particular, asymptotic error rate is 0 if Bayes rate is 0.
- Decision boundary:



Is kNN ideal? ... See Extra



Thank You

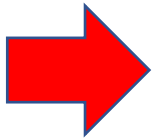
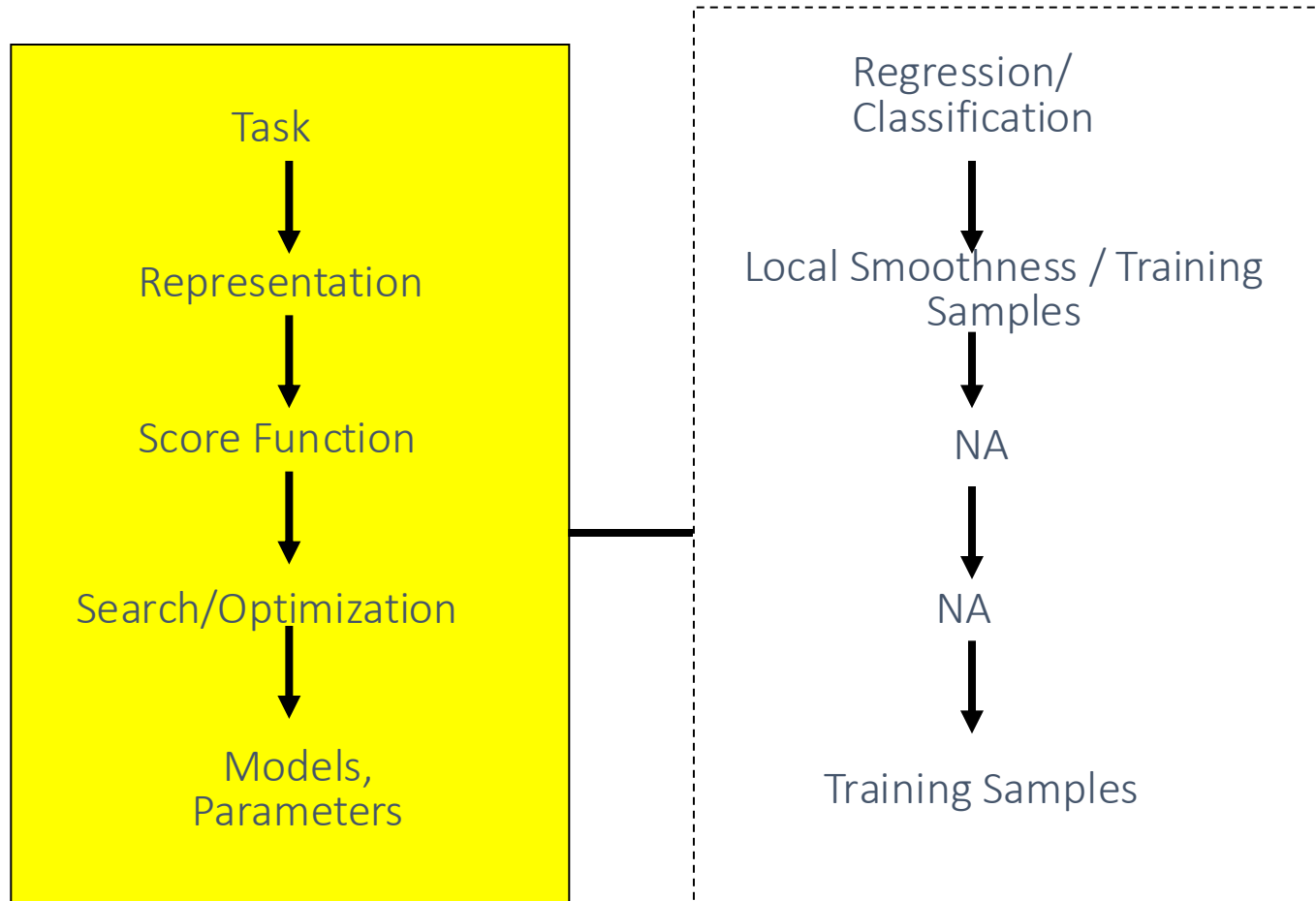


References

- Prof. Tan, Steinbach, Kumar's "Introduction to Data Mining" slide
- Prof. Andrew Moore's slides
- Prof. Eric Xing's slides
- Hastie, Trevor, et al. *The elements of statistical learning*. Vol. 2. No. 1. New York: Springer, 2009.

Extra: More about K-nearest-neighbor

K-Nearest Neighbor



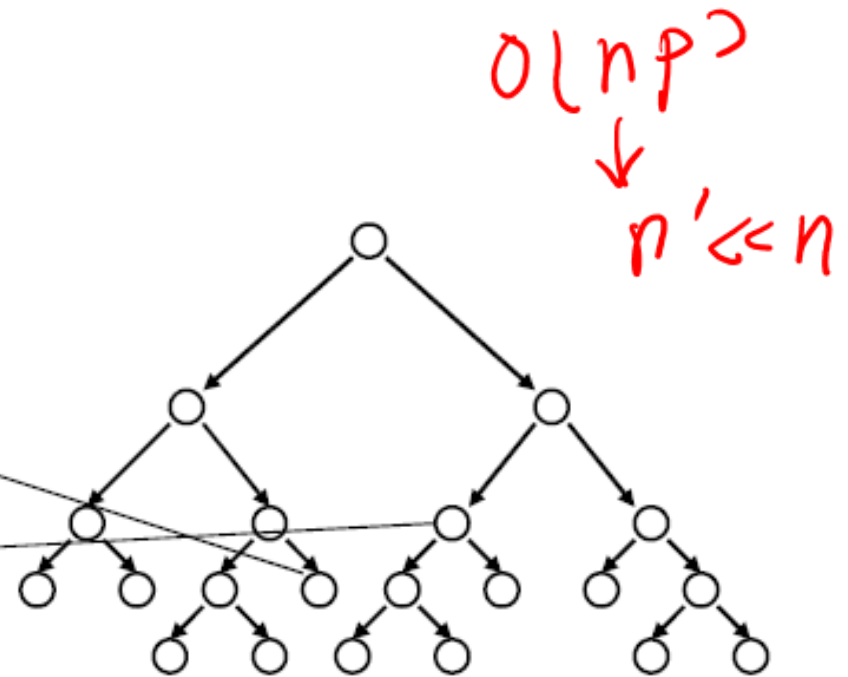
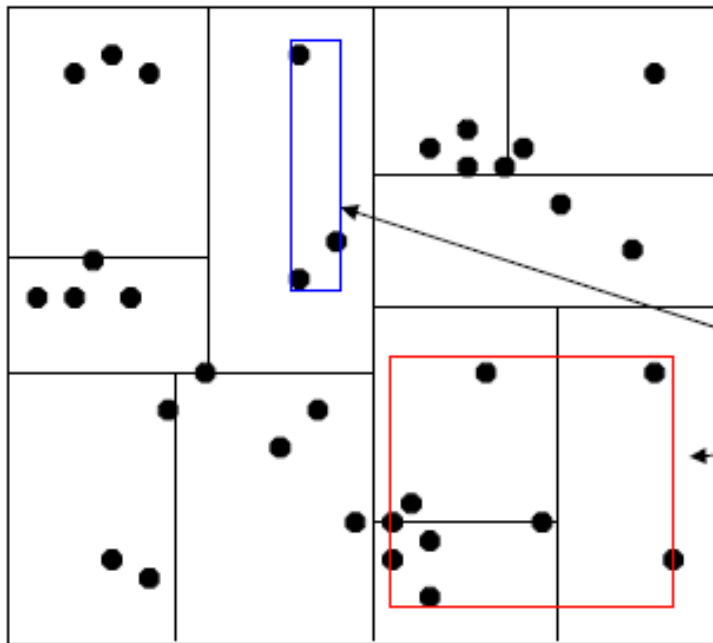
Computational
Scalable?

Computational Time Cost

	Train (n)	Test ($m=1$)
Linear Regression	$O(np^2 + p^3)$	$O(p)$
KNN	$O(1)$	$O(np) +$ $O(\text{sort } n-k)$???

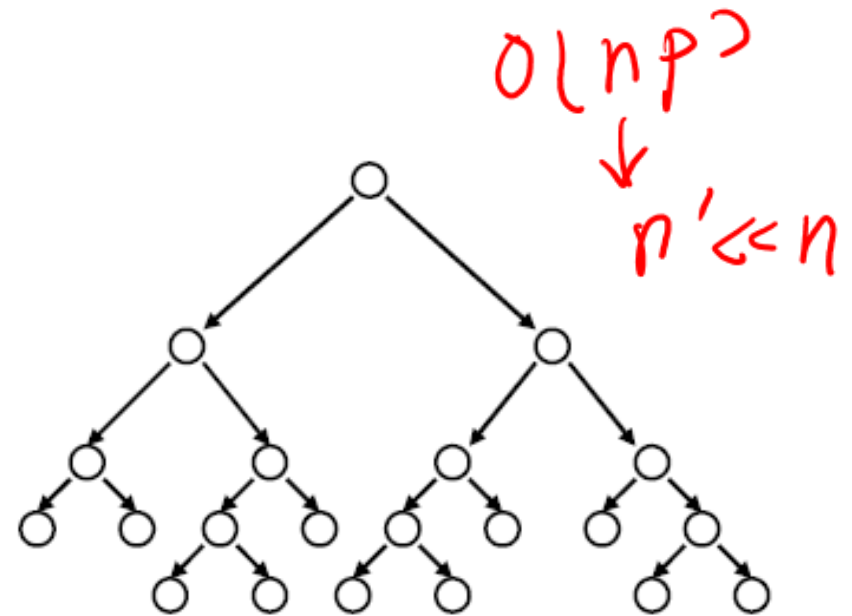
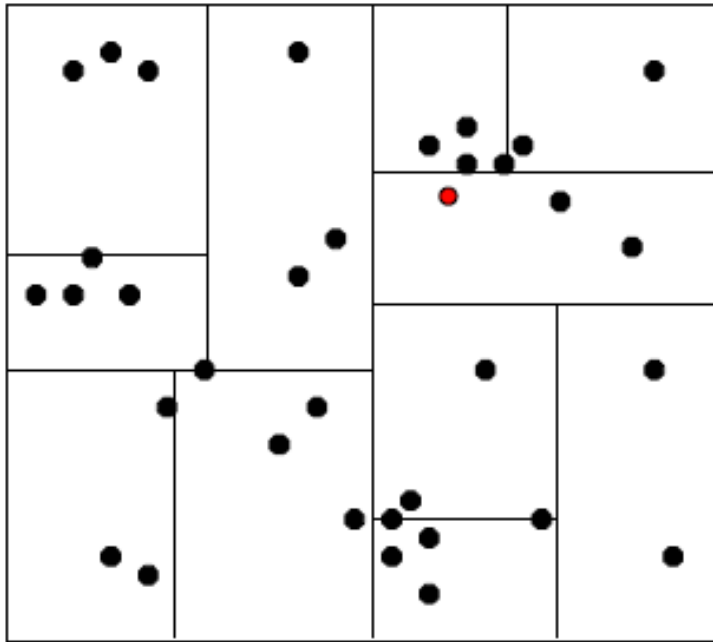
$$p = 30,000$$
$$n = 20,000$$

KD Tree for NN Search



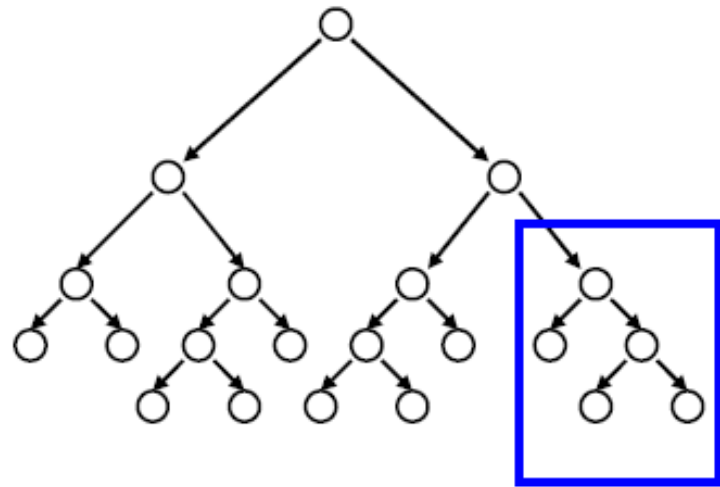
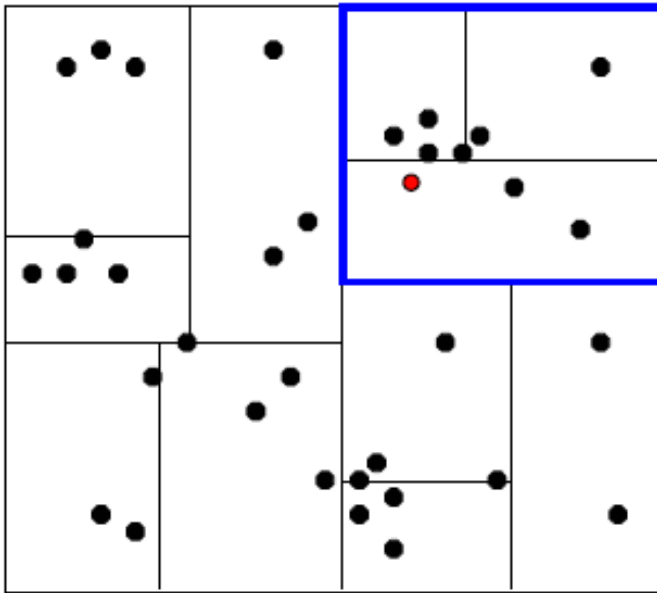
- Each node contains
 - Children information
 - The tightest box that bounds all the data points within the node.

NN Search by KD Tree



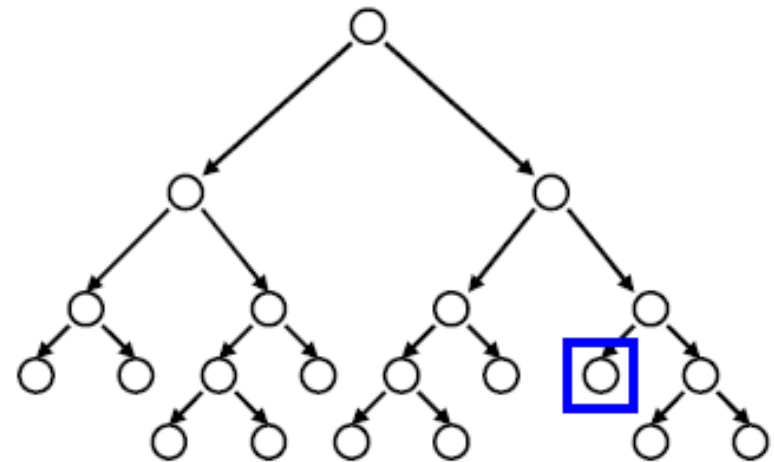
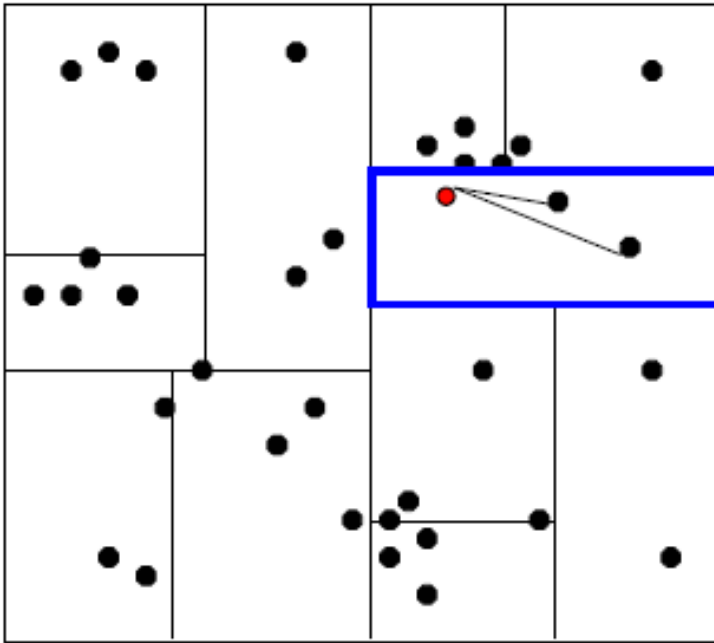
We traverse the tree looking for the nearest neighbor of the query point.

NN Search by KD Tree



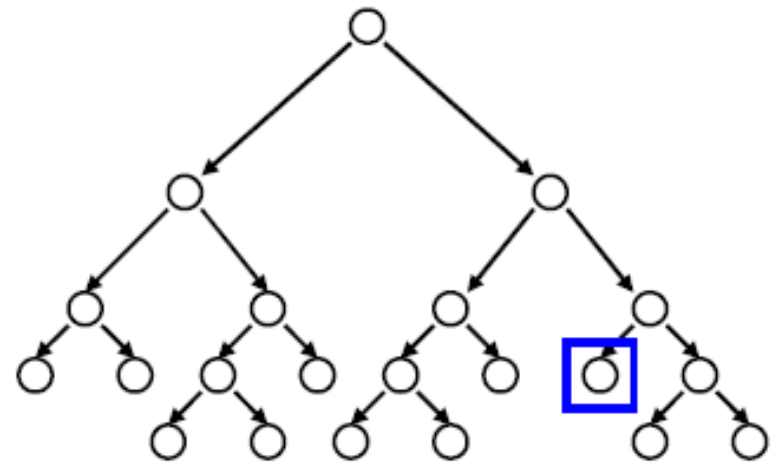
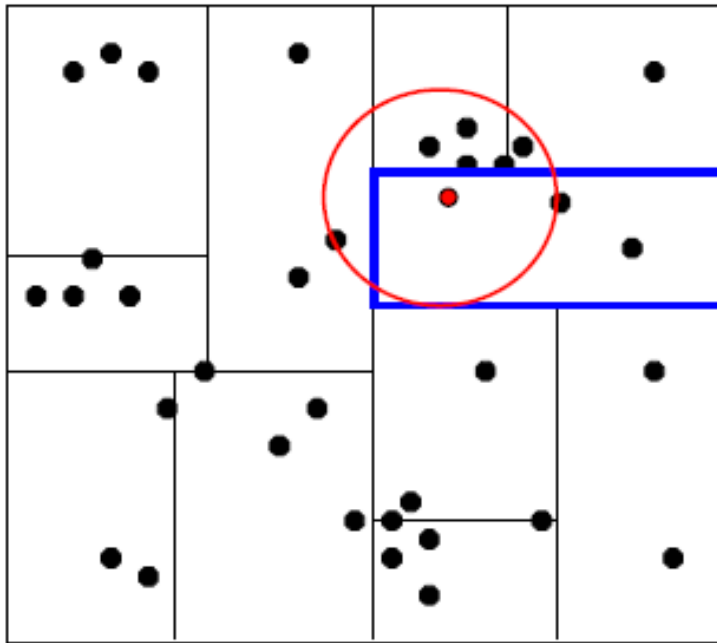
Examine nearby points first: Explore the branch of the tree that is closest to the query point first.

NN Search by KD Tree



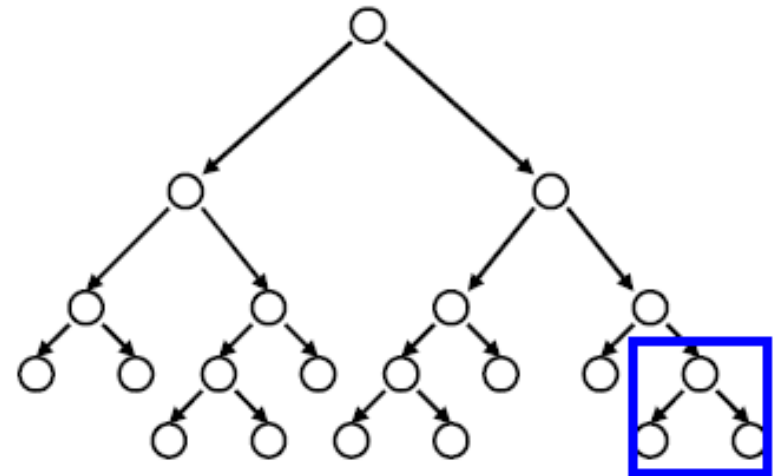
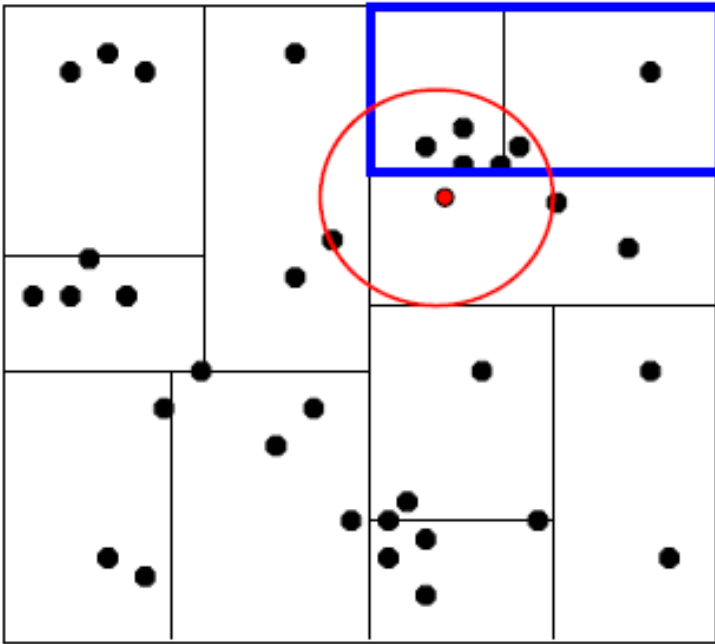
When we reach a leaf node: compute the distance to each point in the node.

NN Search by KD Tree



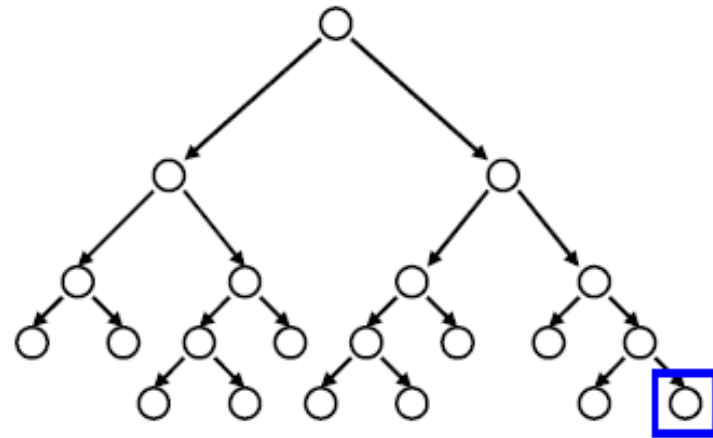
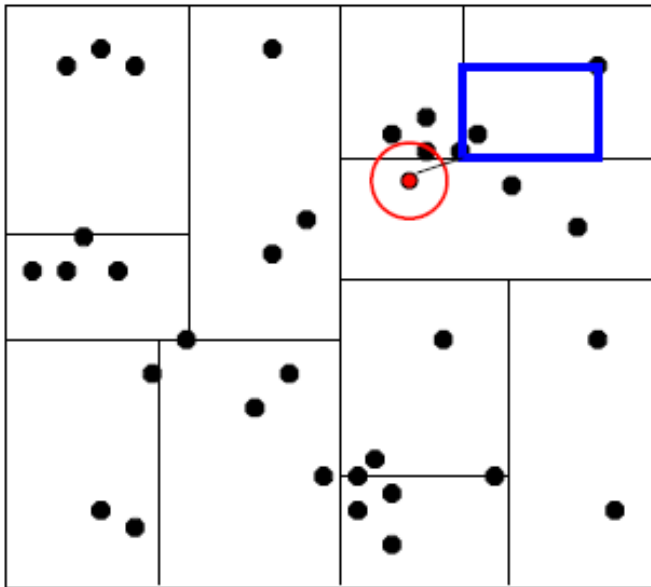
When we reach a leaf node: compute the distance to each point in the node.

NN Search by KD Tree



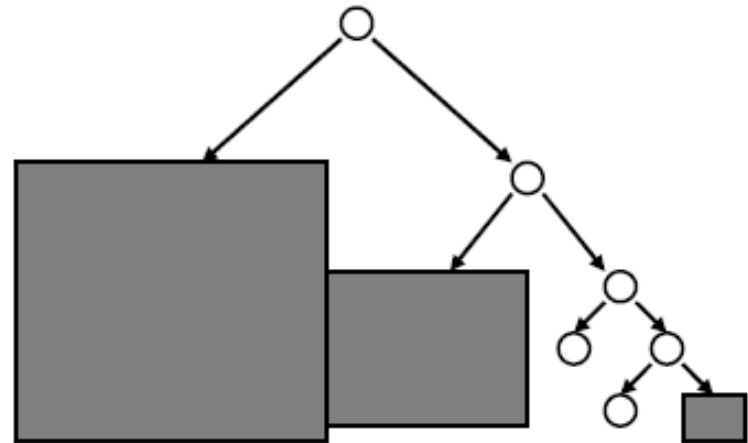
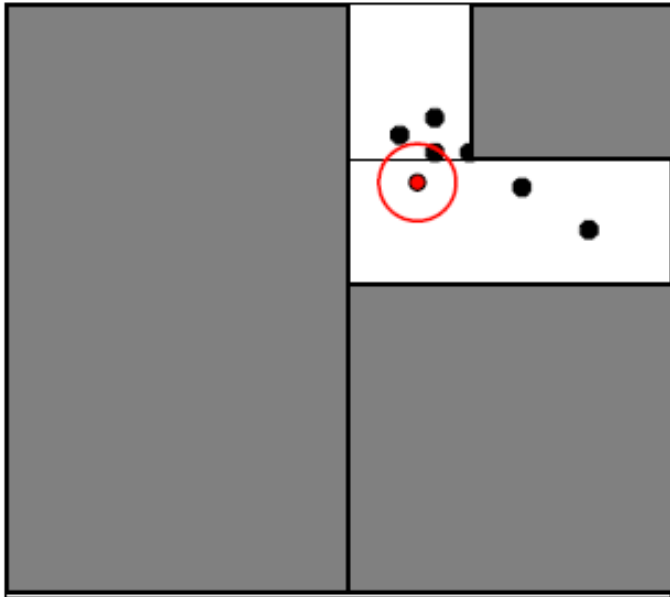
Then we can backtrack and try the other branch at each node visited.

NN Search by KD Tree



Using the distance bounds and the bounds of the data below each node, we can prune parts of the tree that could NOT include the nearest neighbor.

NN Search by KD Tree



Using the distance bounds and the bounds of the data below each node, we can prune parts of the tree that could NOT include the nearest neighbor.

Curse of Dimensionality

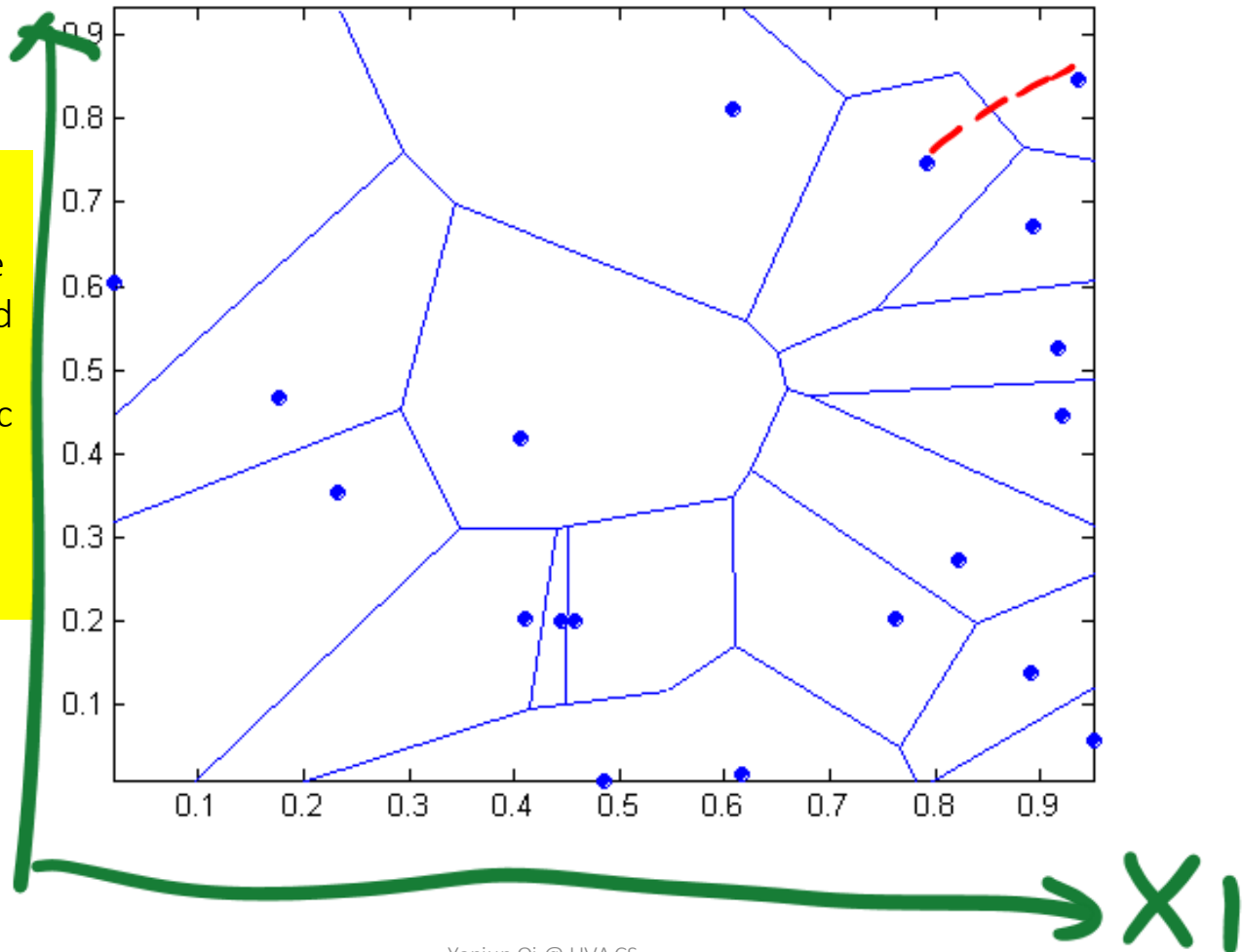
- Imagine instances described by 20 attributes, but only 2 are relevant to target function
- Curse of dimensionality: nearest neighbor is easily misled when high dimensional X
- Consider N data points uniformly distributed in a p -dimensional unit ball centered at origin. Consider the nn estimate at the origin. The mean distance from the origin to the closest data point is:

$$d(p, N) = \left(1 - 2^{-1/N}\right)^{1/p} \approx 1 - \frac{\log N}{p}$$

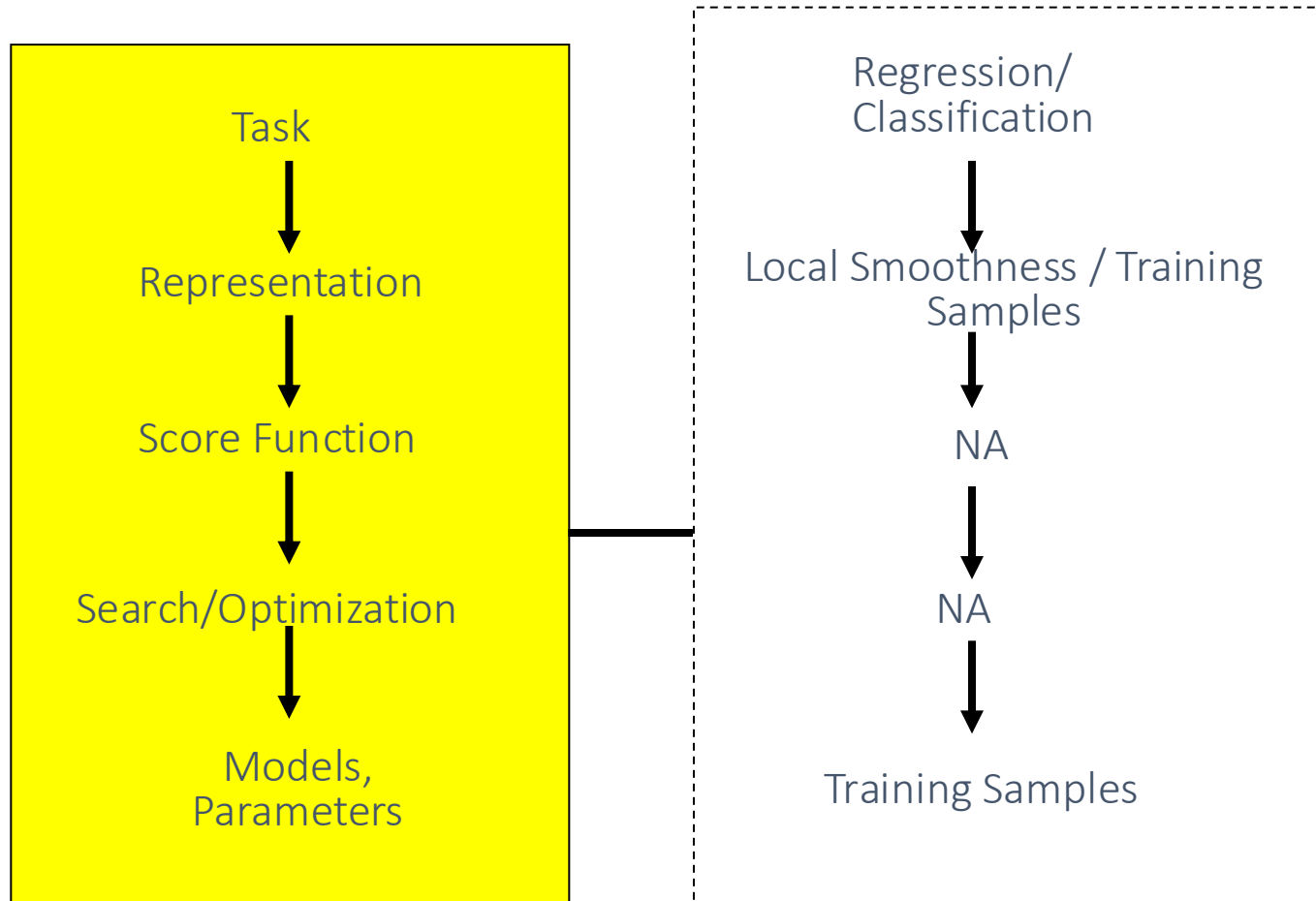
e.g., 1-nearest neighbor

x_2

Voronoi diagram:
partitioning of a plane
into regions based
on distance to
points in a specific
subset of the
plane.

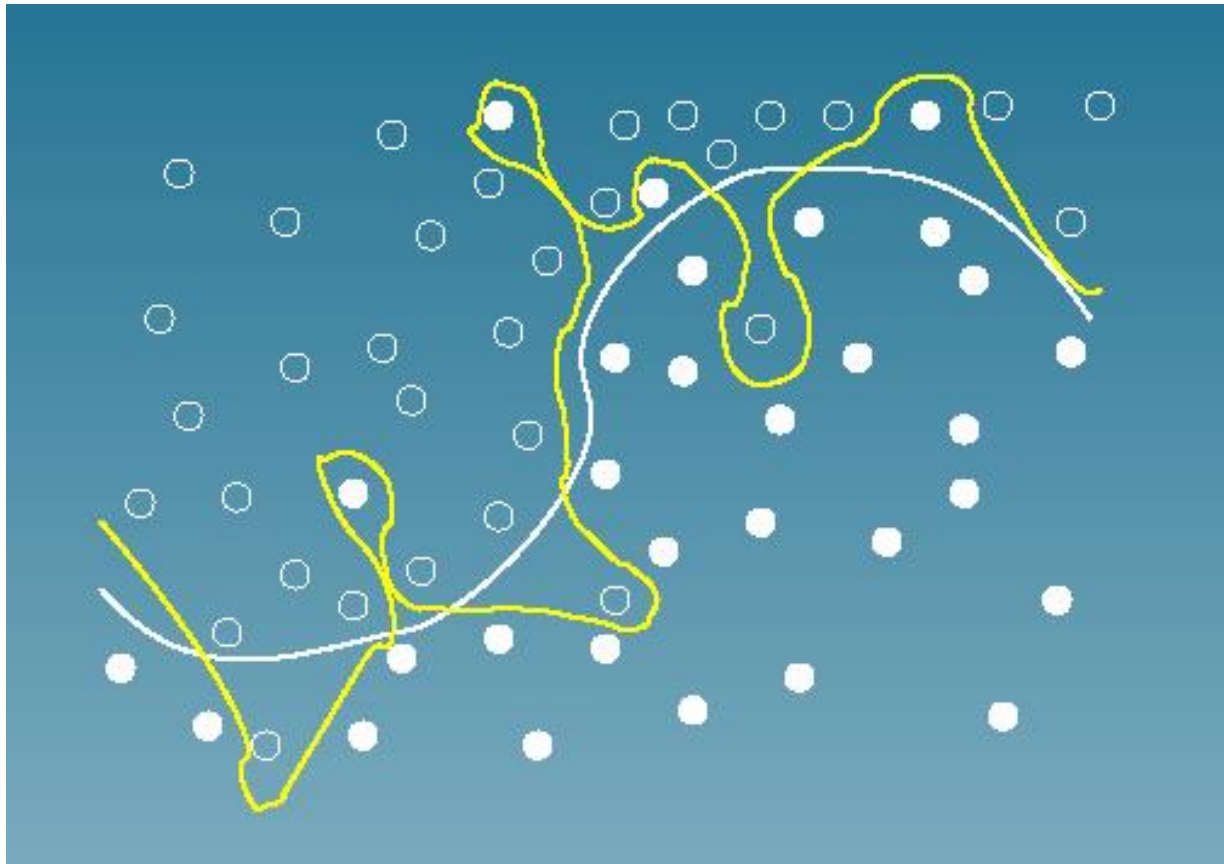


K-Nearest Neighbor



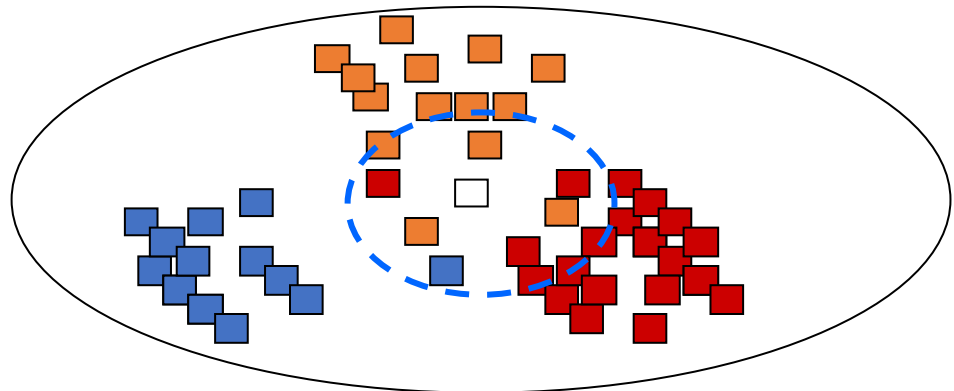
Asymptotically Sound?

Is kNN ideal? ... See Extra



Probabilistic Interpretation of KNN

- Estimate conditional probability $\Pr(y | x)$
 - Count of data points in class y in the neighborhood of x



Bayes Classifier

- ***Bayes classifier*** is the best classifier which minimizes the probability of classification error.
- A classifier becomes the ***Bayes classifier*** if the density estimates converge to the true densities
 - when an infinite number of samples are used
 - The resulting error is the ***Bayes error***, the smallest achievable error given the underlying distributions.

The Bayes Rule

- What we have just did leads to the following general expression:

$$P(Y | X) = \frac{P(X | Y)p(Y)}{P(X)}$$

This is Bayes Rule

Bayes, Thomas (1763) An essay towards solving a problem in the doctrine of chances. *Philosophical Transactions of the Royal Society of London*, **53:370-418**



The Bayes Decision Rule for Minimum Error

- The *a posteriori* probability of a sample

$$P(Y = i | X) = \frac{p(X | Y = i)P(Y = i)}{p(X)} = \frac{\pi_i p_i(X)}{\sum_i \pi_i p_i(X)} \equiv q_i(X)$$

- Bayes Test:

- Likelihood Ratio:

$$\ell(X) =$$

- Discriminant function:

$$h(X) =$$

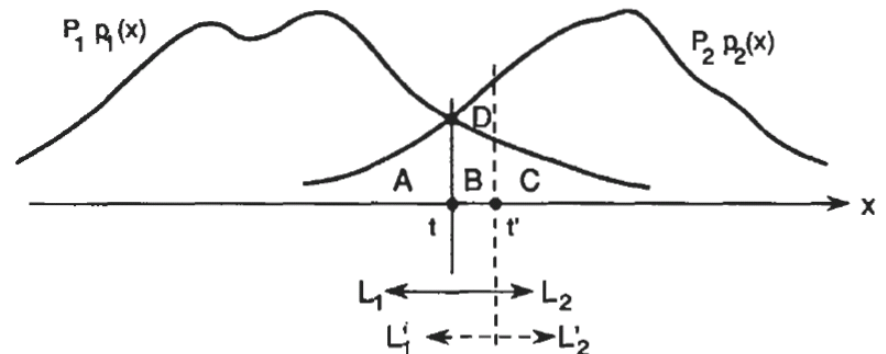
Bayes Error

- We must calculate the *probability of error*
 - the probability that a sample is assigned to the wrong class
- Given a datum X , what is the *risk*?

$$r(X) = \min[q_1(X), q_2(X)]$$

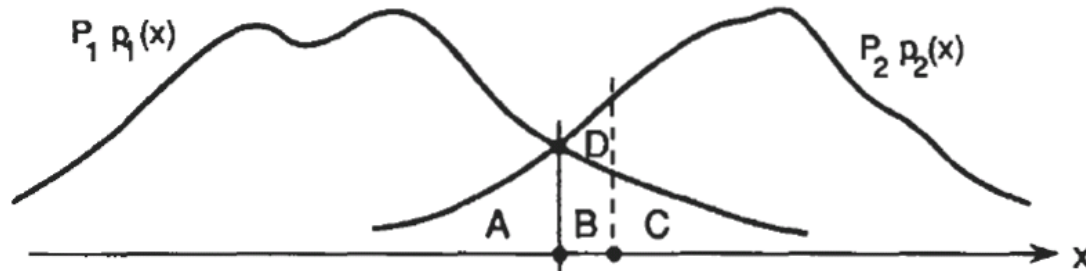
- The Bayes error (the expected risk):

$$\begin{aligned} \epsilon &= E[r(X)] = \int r(x)p(x)dx \\ &= \int \min[\pi_1 p_1(x), \pi_2 p_2(x)]dx \\ &= \pi_1 \int_{L_1} p_1(x)dx + \pi_2 \int_{L_2} p_2(x)dx \\ &= \pi_1 \epsilon_1 + \pi_2 \epsilon_2 \end{aligned}$$



More on Bayes Error

- Bayes error is the lower bound of probability of classification error



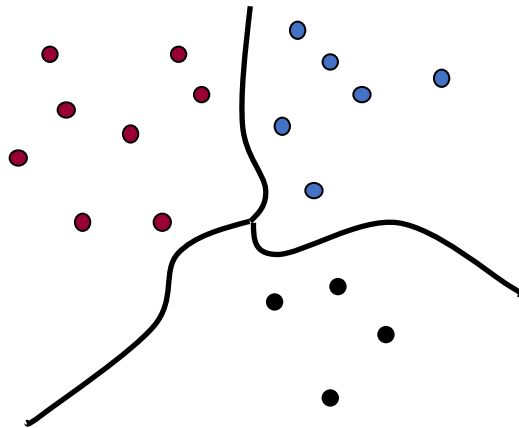
- Bayes classifier is the theoretically best classifier that minimizes probability of classification error
- Computing Bayes error is in general a very complex problem. Why?
 - Density estimation:
 - Integrating density function:

$$\epsilon_1 = \int_{\ln(\pi_1/\pi_2)}^{+\infty} p_1(x) dx$$

$$\epsilon_2 = \int_{-\infty}^{\ln(\pi_1/\pi_2)} p_2(x) dx$$

kNN Is Close to Optimal

- Cover and Hart 1967
- Asymptotically, the error rate of 1-nearest-neighbor classification is less than twice the Bayes rate [error rate of classifier knowing model that generated data]
- In particular, asymptotic error rate is 0 if Bayes rate is 0.
- Decision boundary:



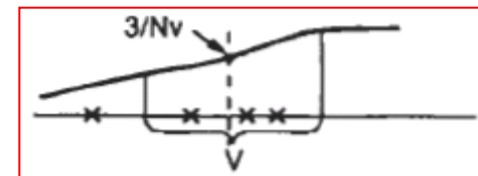
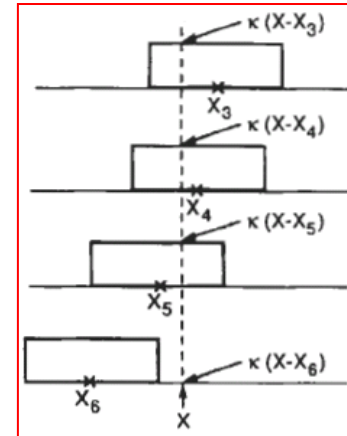
Where does kNN come from?

- How to estimation $p(X)$?
- Nonparametric density estimation
 - Parzen density estimate

E.g. (Kernel density est.):

$$\hat{p}(X) = \frac{1}{N} \sum_{i=1}^N \kappa(X - x_i)$$

More generally: $\hat{p}(X) = \frac{1}{N} \frac{k(X)}{V}$



Where does kNN come from?

- Nonparametric density estimation

- Parzen density estimate $\hat{p}(X) = \frac{1}{N} \frac{k(X)}{V}$

- kNN density estimate $\hat{p}(X) = \frac{1}{N} \frac{(k-1)}{V(X)}$

- Bayes classifier based on kNN density estimator:

$$h(X) = -\ln \frac{p_1(X)}{p_2(X)} = -\ln \frac{(k_1-1)N_2V_2(X)}{(k_2-1)N_1V_1(X)} > \ln \frac{\pi_1}{\pi_2}$$

Voting kNN classifier

Pick K_1 and K_2 implicitly by picking $K_1+K_2=K$, $V_1=V_2$, $N_1=N_2$

Asymptotic Analysis

- Condition risk: $r_k(X, X_{NN})$
 - Test sample X
 - NN sample X_{NN}
 - Denote the event X is class l as $X \ll l$
- Assuming $k=1$

$$\begin{aligned} r_1(X, X_{NN}) &= Pr\left\{\{X \leftrightarrow 1 \ \& \ X_{NN} \leftrightarrow 2\} \text{ or } \{X \leftrightarrow 2 \ \& \ X_{NN} \leftrightarrow 1\} | X, X_{NN}\right\} \\ &= Pr\left\{\{X \leftrightarrow 1 \ \& \ X_{NN} \leftrightarrow 2\}\right\} + Pr\left\{\{X \leftrightarrow 2 \ \& \ X_{NN} \leftrightarrow 1\} | X, X_{NN}\right\} \\ &= q_1(X)q_2(X_{NN}) + q_2(X)q_1(X_{NN}) \end{aligned}$$

- When an infinite number of samples is available, X_{NN} will be so close to X

$$r_1^*(X) = 2q_1(X)q_2(X) = 2\xi(X)$$

Asymptotic Analysis, cont.

- Recall conditional Bayes risk:

$$r^*(X) = \min[q_1(X), q_2(X)]$$

$$= \frac{1}{2} - \frac{1}{2}\sqrt{1 - 4\xi(X)}$$

$$= \sum_{i=1}^{\infty} \frac{1}{i} \binom{2i-2}{i-1} \xi^i(X) \quad \text{This is called the MacLaurin series expansion}$$

- Thus the asymptotic condition risk

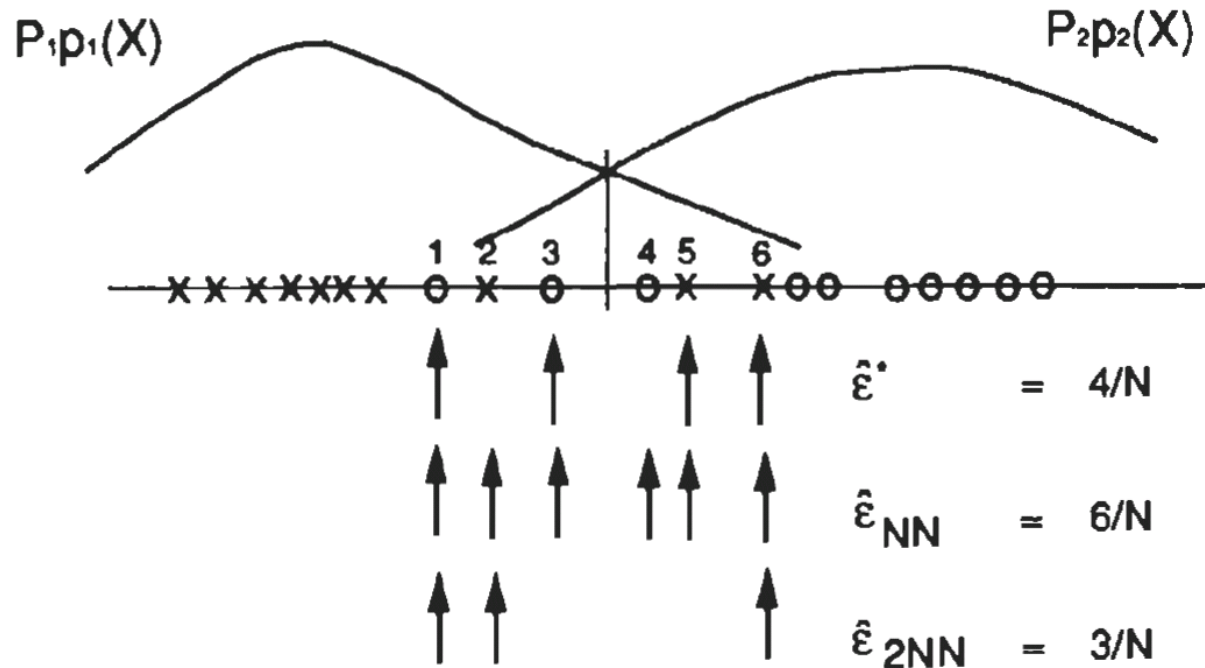
$$r_1^*(X) = 2\xi(X) \leq 2r^*(X)$$

- It can be shown that $\epsilon_1^* \leq 2\epsilon^*$
 - This is remarkable, considering that the procedure does not use any information about the underlying distributions and only the class of the single nearest neighbor determines the outcome of the decision.

In fact

$$\frac{1}{2}\epsilon^* \leq \epsilon_{2NN}^* \leq \epsilon_{4NN}^* \leq \dots \leq \epsilon^* \leq \dots \leq \epsilon_{3NN}^* \leq \epsilon_{NN}^* \leq 2\epsilon^*$$

- Example:



References

- Prof. Tan, Steinbach, Kumar's "Introduction to Data Mining" slide
- Prof. Andrew Moore's slides
- Prof. Jin Rong's slides about kNN
- Prof. Eric Xing's slides
- Hastie, Trevor, et al. *The elements of statistical learning*. Vol. 2. No. 1. New York: Springer, 2009.